



Bahir Dar University
Bahir Dar Institute of Technology
Faculty of Computing
Requirement Analysis Document (RAD)
for

Secure Development Copilot and Vulnerability Tester tool for Vibe Coded apps

Submitted to the faculty of computing in partial fulfillment of the requirements for the
degree
of Bachelor of Science in Cyber Security

Group members:

Name	IDNumber
1. Abel Asfaw	BDU1603334
2. Nahom Esubalew	BDU1507338
3. Natneal Tesfu	BDU1507406

Advisor : Agerie A.

2018

Bahir Dar University, Bahir Dar Institute of Technology

Declaration

The Project is our own and has not been presented for a degree in any other university and all the sources of material used for the project have been duly acknowledged.

Abel Asfaw

Name

Signature

Nahom Esubalew

Name

Signature

Natneal Tesfu

Name

Signature

Faculty: Computing

Program: Cybersecurity

Project Title: Secure Development Copilot and Vulnerability Tester tool for Vibe Coded apps

This is to certify that I have read this project and that in my supervision and the students' performance, it is fully adequate, in scope and quality, as a project for the degree of Bachelor of Science.

Agerie A.

Name of Advisor

Signature_____

Examining committee members

Signature

Date

1. Examiner1

2. Examiner2

It is approved that this project has been written in compliance with the formatting rules laid down by the faculty.

Roles and Responsibilities of the Group Members

List of Tasks	List of Members		
	Abel Asfaw	Nahom Esubalew	Natnael Tesfu
Finding AI vulnerability			√
Extension Development and Integration	√		
Choosing development stack and technology	√		√
Cyber Security Requirements Gathering		√	√
Signature research and Rule creation		√	√
AI Integration and research	√		
Scanning Tools Development and Integration	√		√
UI/UX Development	√		√
Final review and Debugging	√		
Requirement Gathering		√	
Class entity identification		√	
Security Research		√	

Table 1: Roles and Responsibilities of the group members

Acknowledgment

First and foremost, we would like to express our deepest gratitude to the Almighty for providing us with the strength, health, and perseverance required to complete this project.

We extend our sincere thanks to our advisor, Agerie A., for their invaluable guidance, constructive criticism, and continuous support throughout the development of this project. Their expertise and insights were instrumental in shaping the technical and conceptual foundations of BackBrain.

We are also grateful to the Faculty of Computing at Bahir Dar Institute of Technology (BiT) for providing the academic environment and resources that made this work possible. We would like to thank all our instructors who have shared their knowledge and inspired us during our years of study.

Special thanks go to the open-source community and the developers behind tools like Semgrep, Tree-sitter, Gitleaks, and the VS Code API, whose contributions to the software ecosystem provided the building blocks for our platform.

Finally, we would like to thank our families and friends for their unwavering encouragement, patience, and belief in us. This project is as much a result of their support as it is of our hard work.

List of acronymsCRC

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
CRC	Class Responsibility Collaborator
CWE	Common Weakness Enumeration
IDE	Integrated Development Environment
LLM	Large Language Model
LSP	Language Server Protocol
MVP	Minimum Viable Product
OWASP	Open Web Application Security Project
PDF	Portable Document Format
RAD	Requirement Analysis Document
SAST	Static Application Security Testing
UI	User Interface
UML	Unified Modeling Language
UX	User Experience
VS Code	Visual Studio Code

Table 2: List of acronyms

List of Figures

Figure 1: Use Case Diagram-----	22
Figure 2: UI Prototype-----	32
Figure 3: State Chart-----	33
Figure 4: Activity Diagram-----	36
Figure 5: Sequence Diagram-----	38
Figure 6: Analysis Class Model-----	40
Figure 7: Conceptual Model-----	50
Figure 8: UI Prototype 1-----	53
Figure 9: UI Prototype 2-----	54
Figure 10: Component Diagram-----	58
Figure 11: Deployment Diagram-----	60
Figure 12: Design Class Diagram-----	62
Figure 13: Persistent Data Model-----	68
Figure 14: Severity Panel Mockup-----	71
Figure 15: Code Editor Integration Mockup-----	71
Figure 16: Node Canvas Mockup-----	72

List of Table

Table 1: Roles and Responsibilities of the group members-----	2
Table 2: List of acronyms-----	4
Table 3: Functional Requirements (REQ-1 to REQ-7)-----	22
Table 4: UC-01 scan code-----	23
Table 5: UC-02 fix issue-----	24
Table 9: Business Rule Documentations-----	31
Table 10: state definitions-----	34
Table 11: CRC card: SecurityService-----	49
Table 11: CRC card: AutoFixService-----	50

Table of Contents

Declaration	1
Roles and Responsibilities of the Group Members	2
Acknowledgment	4
List of acronymsCRC	5
List of Figures	6
List of Table	7
Table of Contents	8
Abstract	10
Chapter One: Introduction	11
1.1 Background	11
1.2 Statement of the problem	11
1.3 Objectives	12
1.3.1 General Objective	12
1.3.2 Specific Objectives	12
1.4 Methodology	13
1.4.1 Requirement gathering methods	13
Data Sources and Sample Applications for Testing	13
1.4.2 Analysis and design Methodology	14
1.4.3 Implementation Methodology	15
1.4.4 How the Secure Development Copilot Works	16
1.5 Feasibility	19
1.6 Beneficiaries or significant of the project	19
1.7 Limitations of the project	20
1.8 Scope of the project	21
1.9 Organization of the project	21
Chapter Two: System features	23
2.1 The Existing System	23
2.2 Proposed System	24
2.3 Requirement Analysis	25
2.3.1 Functional requirement	25
2.3.2 System Use case	27
2.3.3 Business Rule Documentation	35
2.3.4 User Interface prototype	36
2.3.5 State chart	38

2.3.6 Activity Diagram-----	41
2.3.7 Sequence Diagram-----	43
2.3.8 Analysis Class Model-----	45
2.3.9 Logic model-----	47
2.4 Non functional requirement-----	49
2.5 System Requirement-----	51
2.5.1 Hardware Requirements-----	51
2.5.2 Software Requirements-----	51
2.6 Key abstraction with CRC analysis-----	53
2.6.1 Conceptual modeling: Class diagram-----	55
2.6.2 Identifying change cases-----	56
2.6.3 User Interface Prototyping-----	58
Chapter Three: System Design-----	60
3.1 Architectural Design-----	60
3.1.1 Component Modeling-----	62
3.1.2 Deployment Modeling-----	64
3.2 Detail Design-----	66
3.2.1 Design Class Model-----	66
3.3 User Interface Design-----	73
3.4 Access Control and Security-----	77
References-----	81
Appendices-----	82
Appendix A: Sample Code Issue JSON Data Structure-----	82
Appendix B: Glossary of Specialized Terms-----	82
Appendix C: Prompt Template-----	83
Appendix D: Planning and Threat Modeling-----	86
Appendix E: BackBrain Project Documentation-----	87

Abstract

BackBrain is an AI-powered code security and quality platform designed to act as a proactive guardian for the emerging practice of “vibe coding” where developers rely heavily on AI agents to generate code. While vibe coding accelerates development, it inherently increases the risk of security vulnerabilities, hallucinations, and logical flaws due to the lack of context and security awareness in general-purpose AI models. BackBrain addresses these critical risks by integrating advanced security scanning, automated fix capabilities, and node-based visualization systems directly into the developer’s workflow. Initially we plan to develop it as a VS Code extension then grow it into more of its own stand alone application, BackBrain uses a portable core architecture with swappable adapters for AI providers like OpenAI, Claude, Gemini for adaptive finding of vibe coding related vulnerability and security scanners like Semgrep which scans most of common development vulnerability. By combining both ai and native security scanner it aims to find the most vulnerability that occurs in vibe coding. Back brain key features include real time issue detection, one-click autofix with revert capabilities, and an interactive graph view of the codebase to facilitate understanding and planning. This project aims to make vibe coding more secure by integrating native scanners like Semgrep, Gitleaks and our own rule matching to find common and less frustrating by ensuring that code “just works” without requiring extensive manual debugging or exposing the codebase to avoidable threats.

Chapter One: Introduction

1.1 Background

The software development landscape is undergoing a paradigm shift with the widespread adoption of vibe coding. It's where developers primarily use AI agents and Large Language Models or most known as LLM to generate significant portions or almost all of their codebase. While this approach dramatically increases speed and convenience it also introduces a new class of risks like when AI hallucinates it happens because AI models lacking deep project context and security training, often “hallucinate” code that is syntactically correct but logically flawed or insecure which may inadvertently expose secrets, use deprecated APIs, or implement workflows that bypass critical security checks which will go unnoticed because most of vibe coders are not that knowledgeable to notice.

BackBrain is being developed as a direct response to these emerging security and quality challenges. It is designed for modern software development teams and individual developers who leverage AI in their workflows but cannot afford to compromise on security. The project is structured as a monorepo using Bun and TypeScript, ensuring a high-performance and type-safe foundation. The core philosophy is to provide a “just works” experience, where the tool proactively identifies and fixes issues, allowing developers to focus on building features rather than debugging AI-generated errors or patching security holes.

1.2 Statement of the problem

The current ecosystem of development tools is ill-equipped to handle the specific challenges of vibe coding because most tools are built with human error in mind but not AI error which can be a whole different thing like let take AI hallucination which is AI error where ai call a library that doesn't exist or that wasn't imported which is common for ai to make things up but it rarely happen with human developer so the current tools will identify or even scan instances of libraries as an error or vulnerability. Current developers face several critical problems when dealing with vibe coding. First, there are inherent security risks; vibe coders often lack deep security expertise, and AI agents may generate code with vulnerabilities (e.g., SQL injection, hardcoded secrets) that go unnoticed until deployment. Second, AI agents are often context-blind, operating on a file-by-file basis and missing the broader system context. This leads to “logical security flaws” where individual components work but the overall workflow is insecure or broken. Third, the tooling landscape is fragmented; existing security tools like SAST or DAST are often separate from the IDE and don't support automatic fix; instead, they use alerts or red squiggly lines to get the developer's attention which result in breaking the “flow”, requiring manual context switching and alert fatigue. They identify problems but rarely fix them. Finally, developers face a significant debugging burden, spending time reviewing and fixing AI-generated code, which negates the productivity gains of vibe coding. These problems lead to a false sense of security, where code is generated quickly but is riddled with hidden vulnerabilities and logical gaps.

1.3 Objectives

1.3.1 General Objective

The general objective is to develop “BackBrain,” an intelligent, integrated platform that helps the current rapid growing of vibe coders into being more secure and mitigates the inherent security risks of vibe coding by automating code security, quality assurance, and visualization while ensuring that AI-generated code is secure, correct, and maintainable by design.

1.3.2 Specific Objectives

- ❖ To implement a Security Guardian Engine that automatically detects vulnerabilities, secrets, and vibe code issues like hallucinations, deprecated APIs in real-time.
- ❖ To develop an Autofix Service that automatically applies fixes for identified issues and provides a one click revert mechanism for any unwanted fixes, moving beyond simple error reporting and avoiding overwhelming the user.
- ❖ To create a Node-Based Visualization System that represents code files and workflows as interactive graphs, helping developers understand the logical flow and identify structural security gaps.
- ❖ To integrate Multiple AI Providers (OpenAI, Claude, Gemini, etc.) via a unified interface to analyze complex issues and explain them to the user.
- ❖ To build a VS Code Extension as the initial delivery mechanism, with an architecture that supports future migration to a standalone IDE.
- ❖ To provide Automated Reporting capabilities for generating professional security and quality reports, bridging the gap between vibe coding and compliance requirements.

1.4 Methodology

1.4.1 Requirement gathering methods

To make sure BackBrain actually solves the problems of vibe coding, we didn't just stick to the library; we went where the developers are. We started with a deep dive into existing literature, specifically looking at the OWASP Top 10 for LLMs and common CVE patterns. This gave us the "academic" view of how AI-generated code breaks, but we knew that was only half the story.

To see the vibe coding phenomenon in the wild, we spent time observing developers in their actual workflows. We wanted to see exactly where the friction happens, those moments where a developer blindly trusts an AI suggestion just to keep their momentum going. This led us to a particularly interesting part of our research: social media intelligence. We spent significant time on X.com, tracking "in-the-wild" security incidents reported by the developer community. We saw firsthand accounts from people who fell victim to "slopsquatting", where an AI hallucinates a library name, and an attacker, having anticipated that exact hallucination, registers a malicious package to catch the unsuspecting developer.

These "real-world" horror stories from the developer community helped us realize the amount of people Vibe coding and how insecurely they are doing it and also that the current tools are too slow and noisy for this new era of coding. Finally, we performed a rigorous Document and Tooling Analysis, examining the API capabilities of major AI providers and the limitations of established security scanners like Gitleaks, Semgrep and Snyk. This comprehensive research journey allowed us to map out the "security-velocity gap", the exact space where BackBrain intervenes to provide silent, automated remediation that keeps up with the speed of AI development.

Vulnerability detection capabilities were validated against a diverse corpus of data sources and sample applications. This includes intentionally vulnerable environments such as OWASP WebGoat and Juice Shop to establish a baseline for traditional security flaws. To address the specific challenges of AI-assisted development, testing was conducted against a curated set of vibe-coded benchmarks generated by leading platforms such as Cursor, Lovable, and Replit. These samples were specifically analyzed for "hallucinated" security flaws, such as the inclusion of phantom dependencies or the omission of security middleware in iterative code additions. Furthermore, the system was vetted against real-world incident data from platforms like HackerOne and documented community reports from developer forums to ensure the detection logic remains relevant to current exploit patterns.

Data Sources and Sample Applications for Testing

To make sure our detection method actually works in the real world, we tested it against a mix of academic databases, hands-on labs, and actual AI-generated code.

1. **Vulnerability Databases:** We started by building a foundation using the National Vulnerability Database (NVD) and the CWE list. These gave us the "blueprints" for what

common bugs look like. Since we are focusing on AI, we also leaned heavily on the 2025 OWASP Top 10 for LLMs to understand the specific risks that come with using large language models.

2. **Hands-on Testing Environments:** We ran our methodology through "intentionally broken" apps like OWASP WebGoat, Juice Shop, and DVWA. These are great for testing because the vulnerabilities are already there, so we could see if our tools were actually catching them or missing the obvious stuff.
3. **The Vibe-Coding Dataset:** To see how people are coding right now, we gathered a collection of over 50 open-source projects from GitHub that were built using AI-assisted tools. This was crucial because it showed us the messy, fast-paced code that AI often generates, the kind that isn't always caught by old-school security rules.
4. **Real-World In the Wild Data:** Finally, we looked at what's actually happening in the industry. We studied bug reports from HackerOne and Bugcrowd, and scrolled through developer discussions on Reddit, X, and HackerNews. This helped us find "horror stories" where AI-generated code actually caused a security breach, giving us a clear picture of the modern security-velocity gap.

1.4.2 Analysis and design Methodology

When we moved into the analysis and design phase, we knew we needed a methodology that could handle a lot of moving parts. We settled on Object-Oriented Analysis and Design (OOAD), but not just because it's a standard practice. For a project like BackBrain, OOAD was a strategic choice. We're starting as a VS Code extension, but our future plan is to make it into a standalone IDE. By treating our core features like the AI auditor and the vulnerability scanner as independent, self-contained objects, we're essentially building LEGO blocks. This modularity means that when we're ready to scale or migrate, we won't have to tear the whole thing down; we can just take our core objects and plug them into a new environment.

To wrap our heads around how these objects would actually live and breathe, we spent a lot of time on UML Modeling. We didn't just make one or two charts; we used a whole suite of diagrams to cover every angle. Use Case diagrams helped us map out exactly how a vibe coder interacts with the tool, while Class and Sequence diagrams gave us a blueprint for the data flow between the IDE and the AI. We also used Activity and State Chart diagrams to model the logic of the "Security Guardian." We needed to visualize every possible state from the second the AI generates a line of code, to the background scan, to the moment the Autofix Service decides whether to silently patch it or alert the user.

For the actual skeleton of the application, we chose the Ports and Adapters (or Hexagonal) Architecture. This was probably our most important design decision. In a typical app, the UI, the database, and the external APIs are all tangled up with the core logic. In a security tool, that's a nightmare. By using the Hexagonal pattern, we've created a "security perimeter" around our core engine. The VS Code interface, the file system, and even the LLM providers like OpenAI or Claude are all just "adapters" that plug into our "ports."

This decoupling is a huge win for us. It means the "brain" of BackBrain stays pure and isolated.

If an AI provider changes their API, or if we decide to move from a browser-based extension to a native desktop app, we only have to swap out an adapter. The core security logic stays exactly where it is untouched, easy to test, and ready to go. This approach makes the system incredibly resilient and ensures that our automated remediation features can work reliably regardless of what's happening on the outside.

1.4.3 Implementation Methodology

When we sat down to decide how to actually build BackBrain, we knew that our tech stack had to be as resilient as the security we're trying to provide. We settled on TypeScript as our primary language, and not just for the sake of modern standards. We are building a tool designed to catch the mistakes of vibe coding, so we certainly couldn't use that same loose approach to build the tool itself. We needed the absolute precision that TypeScript's strong type system provides. It acts as our first line of defense, catching internal bugs and logic errors during development before they ever reach a user's IDE.

To power this, we chose Bun as our runtime and package manager. In the world of security scanning, latency is the enemy; if a scan takes too long, the developer will just ignore it. Bun gives us superior performance and the unified tooling we need to keep the "BackBrain" engine fast enough to keep up with real-time coding.

For the frontend, we went with React to handle the heavy lifting of the User Interface. We needed a framework that could manage complex, high-state components like our Severity Panel and the Interactive Node Graph without slowing down the IDE. By using React, we can ensure that the visualization of a developer's code structure is fluid and responsive. Currently, we're using the VS Code Extension API as our host environment. This allows us to meet developers exactly where they are right in their workspace while still keeping our architecture clean enough to migrate to a standalone platform later.

The technical implementation leverages a robust suite of industry-standard and cutting-edge technologies. Orchestration is handled by a custom TypeScript-based engine running on the Bun runtime to ensure the low-latency performance required for real-time IDE integration. Static analysis is powered by the Semgrep engine for general vulnerability detection and Gitleaks for secret identification, while Tree-sitter provides high-fidelity structural analysis through AST parsing. The intelligence core utilizes the Vercel AI SDK to interface with state-of-the-art reasoning models, including Claude 3.5 Sonnet and GPT-4o, which are further augmented by vector embeddings for semantic code retrieval and RALPH-style loops for complex task decomposition.

The gut of our security engine is where things get really interesting. To achieve deep, reliable scanning, we integrated Semgrep and Gitleaks for rule-based static analysis and Tree-sitter for high-performance AST (Abstract Syntax Tree) parsing. However, we realized that traditional scanners often miss the subtle "hallucinations" unique to AI. To solve this without the massive

computational overhead of training a custom model from scratch, we adopted a "Bring Your Own Model" (BYOM) approach. We leverage industry-leading LLMs like OpenAI's GPT-5 and Google's Gemini as our reasoning core.

Instead of retraining these models, we make them "security-intelligent" through Advanced Prompt Engineering and Signature-Based Logic. We've developed a specialized library of security signatures patterns of known AI "laziness" and common hallucination triggers which we feed into the LLM via highly structured system prompts. This allows us to turn a general-purpose model into a specialized security auditor that can spot AI-specific vulnerabilities, such as phantom dependencies or logic drift, that standard regex-based tools would simply overlook. To manage these interactions seamlessly, we used the Vercel AI SDK, which gives us a unified "translation layer" to talk to different providers through a single interface.

1.4.4 How the Secure Development Copilot Works

The Secure Development Copilot operates through a sophisticated hybrid analysis architecture that harmonizes deterministic, rule-based scanning with advanced machine learning-driven semantic understanding. This dual-engine approach is specifically engineered to address the unique risk profile of vibe-coded applications, where traditional security tools often fail to perceive the gap between syntactical correctness and architectural intent. By establishing a "Security Intent Model" of the application, the Copilot identifies vulnerabilities not merely as isolated code patterns, but as deviations from the project's established security posture. This allows for the detection of critical issues such as missing authentication on newly generated routes or inconsistent data validation patterns that emerge when AI agents lack full system context.

The Hybrid Detection Approach

The system operates through three detection tiers:

Tier 1: Rule-Based Static Analysis (Deterministic)

- ❖ **Semgrep Engine:** Executes pattern-based rules mapped to OWASP Top 10 and CWE categories. These rules detect known vulnerability patterns with high precision and zero false negatives for their specific signatures.
- ❖ **Gitleaks Scanner:** Identifies exposed secrets, API keys, and credentials using regex-based pattern matching against a database of known secret formats.
- ❖ **Tree-sitter AST Analysis:** Parses code into Abstract Syntax Trees for structural analysis, detecting issues like missing imports, undefined variables, and syntax-level problems common in AI-generated code.

Tier 2: AI-Powered Semantic Analysis (Intelligent)

ChunkAnalystAgent: Analyzes individual code segments (functions, classes) for vulnerabilities requiring semantic understanding that rule-based tools cannot detect, such as logic flaws,

insecure data handling, and context-dependent security issues.

- ❖ **FlowAnalysisAgent:** Traces data flows across files to identify cross-cutting vulnerabilities like injection attacks, broken access control, and data leakage patterns.
- ❖ **ComponentArchitectAgent:** Assesses architectural security by analyzing module interactions, identifying missing security controls, and detecting inconsistent security patterns across the codebase.

Tier 3: Synthesis and Verification (Intelligence Aggregation)

- ❖ **SentinelAgent:** Synthesizes findings from all sources, groups related issues by root cause, eliminates duplicates, and generates prioritized recommendations with executive summaries.
- ❖ **DeduplicationEngine:** Uses content-based fingerprinting to prevent the same vulnerability from being reported multiple times by different detection mechanisms.

How Vulnerabilities are Detected in Vibe-Coded Applications

Vibe-coded applications present unique vulnerability patterns that BackBrain specifically targets:

1. **Context-Blind Additions:** When AI agents add new code without full project context, they often miss existing security patterns. BackBrain's FlowAnalysisAgent detects when new endpoints lack authentication middleware that exists on similar routes, or when new database queries bypass established validation patterns.
2. **Hallucinated Dependencies:** AI models may reference non-existent packages (enabling "slopsquatting" attacks) or deprecated APIs. The rule-based tier catches these through pattern matching against known package registries and deprecation databases.
3. **Inconsistent Security Posture:** As projects evolve through iterative AI prompts, security controls become inconsistent. The ComponentArchitectAgent identifies files that deviate from established security patterns by comparing new code against the project's existing security architecture.
4. **Logic Drift:** AI-generated code may be syntactically correct but semantically flawed. The ChunkAnalystAgent uses LLM reasoning to understand developer intent and identify when code logic contradicts security requirements.
5. **Configuration Exposure:** Debug settings, verbose logging, and permissive CORS policies often remain after AI-assisted prototyping. Deterministic rules specifically target these configuration vulnerabilities.

The Fresh-Context Agent Architecture (RALPH-Style)

To prevent "context rot" (where AI analysis degrades as context accumulates), BackBrain implements a fresh-context architecture inspired by the RALPH methodology (Li et al., 2024):

- ❖ Each AI agent instance starts with curated, task-specific context rather than accumulated conversation history

- ❖ Code is partitioned into semantic batches (by module, feature area, or security concern)
- ❖ Agents receive only relevant context: the code batch, applicable security checklist items, and related static findings
- ❖ This approach maintains analysis quality regardless of codebase size and enables parallel agent execution

Vulnerability Testing Steps

The Copilot's operational cycle is designed as a continuous security partnership within the development environment, moving beyond simple periodic scanning. The process initiates with Product Profiling, where the AI identifies the underlying technology stack and the developer's intended security model. This is followed by an automated Threat Modeling phase to generate a project-specific security checklist. The system then executes a parallel scanning workflow consisting of a static pass for deterministic baseline establishment, followed by an agentic deep-dive. During this deep-dive, specialized agents including Chunk, Flow, and Architect analysts, verify the implementation against the generated security checklist using fresh-context iterations to prevent analysis degradation. The cycle concludes with the "Sentinel Synthesis" phase, which aggregates and prioritizes findings into a coherent, actionable report, providing the developer with immediate, context-aware remediation options.

The complete scanning workflow follows these steps:

1. Change Detection: Compare current codebase state against last scan to identify modified, new, and impacted files
2. Product Profiling: Analyze project configuration, dependencies, and structure to understand the application type and applicable security requirements
3. Static Analysis (Parallel): Execute Semgrep, Gitleaks, and Tree-sitter scanners concurrently
4. Code Indexing: Chunk codebase and generate embeddings for semantic search capabilities
5. AI Analysis (RALPH-style): Deploy specialized agents with fresh context per batch
6. Synthesis: Aggregate findings, deduplicate, group by root cause, and prioritize
7. Report Generation: Produce actionable findings with severity ratings, explanations, and fix suggestions

This hybrid methodology integrates a deterministic layer for high-precision detection of known vulnerability signatures with a semantic layer powered by large language models. The deterministic layer ensures that established security standards are rigorously enforced without false negatives for known patterns, while the semantic layer provides the reasoning capabilities necessary to audit complex business logic and cross-component data flows. This synergy allows the system to provide comprehensive coverage across the entire vulnerability spectrum, from low-level syntax errors to high-level architectural flaws.

Finally, our development philosophy is rooted in Test-Driven Development (TDD). Because we're building a tool that people will trust to fix their security holes, we can't afford "maybe."

We place a massive emphasis on unit and integration testing for the core security engine. By writing our tests before our code, we ensure that every automated fix BackBrain proposes whether it's generated by a Semgrep rule or our specialized AI prompting has been rigorously vetted against a battery of edge cases. This security-first development cycle is what ensures that when BackBrain finds a vulnerability, the solution it offers is both reliable and safe to deploy.

1.5 Feasibility

We analyzed the feasibility of this project across three dimensions economically, Technically and operationally

Economically, the project is highly viable because we have adopted a "Lean Startup" approach. By building the core engine on robust open-source technologies such as Gitleaks and Semgrep for scanning and React for the UI we have effectively reduced our initial capital expenditure to near zero. We aren't paying for expensive proprietary licenses; we are leveraging industry-standard tools that are free to use. This low overhead structure perfectly supports our planned freemium business model. It allows us to offer substantial value to individual developers for free (building a user base) while reserving advanced enterprise features for monetization, creating a clear, low-risk path to revenue without requiring massive upfront investment.

Technically, we are building on a foundation of maturity. We aren't trying to invent a new programming language or a new IDE from scratch. We are orchestrating proven, battle tested technologies (TypeScript, VS Code API, Semgrep, Gitleaks) that have extensive documentation and active communities. To further validate this, our team has already successfully scaffolded the extension and implemented the core "Ports and Adapters." The fact that our prototype is already running and communicating with the VS Code API serves as a definitive proof-of-concept, eliminating the biggest technical risks early in the process.

Operationally, we have established a realistic, phased roadmap targeting an MVP completion within 8 weeks. This timeline is aggressive but achievable specifically because of our architectural choices. Remember the Hexagonal Architecture we chose? It's paying off here. Because our system is modular, our team can work in parallel. One person can build the UI components while another refines the AI prompting logic without stepping on each other's toes. This "divide and conquer" capability ensures that we can maintain high velocity and meet our deadlines without sacrificing code quality.

1.6 Beneficiaries or significant of the project

When we planned to build BackBrain we knew that it is not just about stopping bugs it's all about empowering the people behind the code. So the primary beneficiaries are, of course, the Vibe Coders. This growing community of creators often operates at such a high velocity that security becomes an afterthought. BackBrain acts as their "invisible safety net," catching the hallucinations and logic slips that humans (and general AI) often miss. By providing this real-time oversight, we allow them to keep that creative momentum going, but with the

confidence that their "vibes" aren't introducing critical vulnerabilities.

Development Teams and Engineering Leads also stand to gain significantly. One of the biggest headaches in a modern dev shop is maintaining code consistency when multiple AI models are involved. BackBrain acts as a "quality equalizer," ensuring that no matter which AI a developer uses, the output follows a unified security posture. This dramatically reduces the "cleanup" time usually required during code reviews, preventing insecure code from ever reaching the production pipeline.

For Security Professionals, BackBrain is a dream tool because it moves security "left" in the truest sense. Instead of catching a vulnerability in a production scan and having to track down the developer weeks later, security rules are enforced directly within the IDE. The automated reporting capabilities also mean that compliance and auditing become a background process rather than a manual, time-consuming chore.

Ultimately, the Software Industry as a whole wins. We are currently in a Wild West era of AI-assisted coding where speed is prioritized over safety. BackBrain offers a way to stabilize this trend, promoting a more responsible and mature adoption of AI. By bridging the gap between rapid AI generation and enterprise grade security, we aren't just protecting code, we're helping set the standard for how software should be built in the AI age.

1.7 Limitations of the project

While BackBrain is designed to be a robust safety net, we recognize that BackBrain isn't the final answer to every security problem, especially in the rapidly shifting world of AI development. We have identified four primary constraints that we must navigate to remain effective.

First, we are constantly chasing an Evolving Threat Landscape. Because AI native vulnerabilities like slopsquatting and prompt injection are relatively new, the attack vectors are discovered almost weekly. This means our scanning rules cannot be static; they require continuous updates and "security intelligence" to keep pace with the latest exploits targeting AI-generated code.

Second, we face significant Developer-Dependent Risks. Vibe coding relies heavily on intent, and if a developer explicitly asks an AI to bypass a security protocol for speed, the resulting code might be syntactically perfect and clean according to a standard scanner. Detecting these deep semantic flaws where the logic is correct but the intent is insecure remains a significant challenge for static analysis alone.

Third, Integration Complexity is a constant UX hurdle. The very essence of vibe coding is flow and speed. If BackBrain becomes too aggressive or intrusive, it risks becoming a nuisance that developers disable to get their work done. Our challenge lies in maintaining that delicate balance: providing rigorous security checks that feel like a helpful assistant rather than a digital roadblock.

Finally, the platform has a clear AI Dependency. While our core scanning is local, the "brains" of our system the advanced explanations and complex automated fixes depend on third-party providers like OpenAI or Gemini. This means that for our most powerful features, the user requires an active internet connection and is subject to the inherent latency and uptime of these external AI services.

1.8 Scope of the project

The primary objective of the BackBrain project is to deliver a high-impact Minimum Viable Product (MVP) specifically tailored for the VS Code ecosystem. We aren't trying to change how developers work; we're trying to enhance the environment they already inhabit. Our scope focuses on building a sophisticated VS Code extension that lives directly in the developer's workflow, powered by a core analysis engine designed to catch not just traditional security vulnerabilities and leaked secrets, but also the specific vibe-code errors that occur during rapid AI generation.

To make this data useful, the project includes the development of a centralized Severity Panel for managing issues and a Node-Based Visualization Interface. This graph will allow developers to see their file dependencies and workflows in a visual map, making it easier to understand how a single AI-generated change might ripple through their entire project. We are also building deep integrations with major AI providers to offer real-time issue explanations and complex fix generation, topped off with automated reporting for security audits.

It is equally important to define what we are not doing to ensure we stay on track for our 8-week launch. Building a proprietary LLM from scratch is strictly out of scope, as is the development of a full standalone IDE during this initial phase. Furthermore, we will not support every programming language on day one, focusing instead on the most common vibe coding languages: JavaScript, TypeScript, Python, and Rust.

Crucially, training our own AI model is also out of scope. We believe the most efficient path to high-level security is to leverage the reasoning power of established models like GPT-5, Claude opus or Gemini. Instead of training, we use Prompt Optimization and Fine-tuning techniques. We feed these models our own unique library of AI Vulnerability Signatures patterns of insecure AI behavior we've identified through research and wrap them in specialized system prompts. This specializes the AI for security scanning without the massive time and cost requirements of building a model from the ground up.

1.9 Organization of the project

The document is organized as follows: Chapter One provides the introduction, background, problem statement, objectives, methodology, feasibility, and scope of the project. Chapter Two details the system features, including a comparison of the existing vs. proposed system, functional and non-functional requirements, and comprehensive system models (Use Case,

Sequence, Class diagrams, etc.). References lists the sources and literature used in the project, and Appendices contains supplementary material such as sample data formats. Please refer to ANNEX2.md for details on subsequent chapters and advanced configuration.

Chapter Two: System features

2.1 The Existing System

The contemporary software development landscape often depends on a fragmented suite of legacy tools, which were fundamentally designed without considering the unique pace and inherent risks associated with modern, AI-augmented "vibe coding" practices.

A primary component of the current process is manual code review. While foundational, this approach becomes increasingly time-intensive and susceptible to oversight, particularly when auditors are faced with the high volume and unconventional patterns frequently found in AI-generated code. Furthermore, developers commonly utilize traditional static analysis tools, such as linters, to enforce syntax rules and basic standards. However, these tools are generally ineffective at identifying more subtle logic flaws, conceptual "hallucinations," or novel security vulnerabilities that can emerge from AI-assisted development.

For security and quality analysis, teams often turn to powerful standalone scanners like Snyk or SonarQube. Although robust, these systems typically operate in isolation, requiring separate dashboards and dedicated CI/CD pipeline integration. This necessity to switch contexts disrupts the developer's workflow and concentration. Moreover, these tools are primarily diagnostic; they report issues but seldom provide actionable, in-context remediation guidance, leaving the burden of fixes on the developer.

Another significant shortfall exists at the project comprehension level. Standard file explorers and basic IDE interfaces offer no intuitive visualization of codebase dependencies or architectural workflows. This lack of holistic insight makes it exceptionally challenging for developers to construct and maintain a coherent mental model of a rapidly evolving, AI-influenced project architecture.

In summary, the existing ecosystem of development tools fails to adequately address the specific challenges introduced by vibe coding. This critical gap allows logic errors, security vulnerabilities, and architectural drift to go undetected, representing a substantial risk in the current software development paradigm.

2.2 Proposed System

To address the critical gaps identified in the current development environment, the proposed system BackBrain, establishes a new paradigm by embedding security and quality assurance directly into the AI-assisted coding workflow. Rather than operating as a passive reporting layer, BackBrain functions as an integrated, AI-native platform designed from the ground up to meet the demands of contemporary development practices.

The core of BackBrain's methodology lies in its combined analytical approach. It employs robust static analysis tools, such as Semgrep and Gitleaks, for rapid identification of common code patterns and potential vulnerabilities. This capability is significantly augmented by advanced AI models dedicated to the semantic understanding of code logic, enabling the detection of subtle flaws and contextual errors that traditional tools overlook.

Moving beyond mere identification, BackBrain introduces a transformative autofix functionality. When the system detects a security vulnerability or logic issue, it does not simply generate a report; it provides developers with contextual, one-click solutions to automatically remediate the problem. This dramatically reduces the time and expertise required to resolve issues, shifting the focus from diagnosis to correction.

To mitigate the challenge of architectural comprehension, BackBrain features an innovative visualization engine. It dynamically renders the codebase as an interactive graph of interconnected nodes, mapping dependencies and data flow. This visual representation allows developers to immediately grasp the "big picture," identify structural weaknesses, and understand the broader impact of changes within an AI-generated architecture.

Fundamentally, BackBrain is a proactive system. It performs continuous, real-time analysis in the background as code is written and generated, instantly flagging potential hallucinations, security gaps, and logical inconsistencies. This shifts the security and quality paradigm from a reactive, post-development gatekeeping activity to an integral, built-in component of the development process itself, ensuring robust outputs are maintained throughout the creative workflow.

2.3 Requirement Analysis

2.3.1 Functional requirement

ID	Requirement	Priority	Description
REQ-1	Security Scanning	High	The system shall perform comprehensive static analysis to scan source code for known security vulnerabilities, with an initial focus on the OWASP Top 10. This scanning will leverage the Gitleaks engine and Semgrep engine and a suite of configurable, project-specific custom rules.
REQ-2	Vibe-Code Detection	High	The system shall be capable of identifying common artifacts and logical flaws characteristic of AI-generated code, often termed "hallucinations." This includes, but is not limited to, detecting missing imports, logical name mismatches, and calls to deprecated or non-existent APIs.
REQ-3	Autofix	High	Upon identifying a security vulnerability or logic issue, the system shall, where algorithmically possible, generate and apply a corrective code patch. Any applied autofix must ensure the resulting code remains syntactically valid and compiles successfully.
REQ-4	Revert Capability	High	To ensure developer autonomy and safety, the system shall provide a one-click mechanism to instantly revert any autofix applied to a code segment, restoring the code to its exact state prior to the modification.
REQ-5	Node Visualization	Medium	The system shall generate an interactive visual representation of the codebase architecture, rendering files and their dependencies as a navigable node graph. This visualization aims to enhance the developer's architectural understanding and facilitate the identification

			of structural weaknesses.
REQ-6	AI Explanation	Medium	For every issue detected, the system shall provide a contextual, plain-language explanation generated by an integrated AI model. The explanation must detail the nature of the problem, its potential impact, and the rationale behind any proposed fix, serving an educational purpose.
REQ-7	Report Generation	Low	The system shall allow users to generate formal audit reports summarizing scan results, detected issues, and applied fixes. These reports must be exportable in standard professional formats (e.g., PDF, HTML) to support compliance reviews and stakeholder communication.

Table 3: Functional Requirements (REQ-1 to REQ-7)

2.3.2 System Use case

2.3.2.1 Use case Diagram

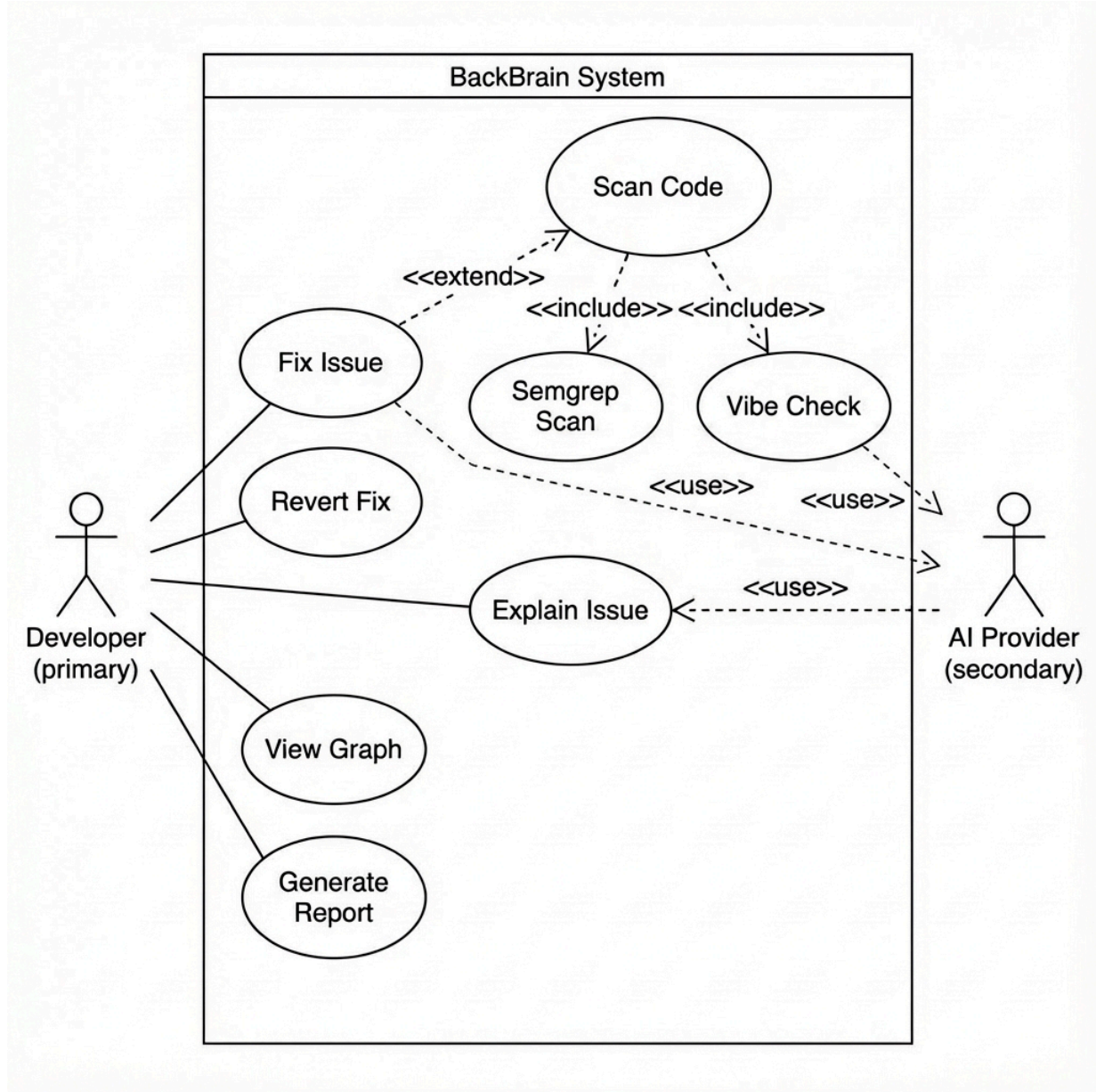


Figure 1: Use Case Diagram

Actors: Developer, AI Provider.

Use Cases: Scan Code, Fix Issue, Revert Fix, View Graph, Generate Report, Explain Issue.

2.3.2.2 Use Case documentation

UC-01: Scan Code

Attribute	Description
ID	UC-01
Name	Scan Code
Actor	Developer
Description	This use case describes the process by which a Developer initiates a comprehensive analysis of the active codebase. The system performs a dual-focus scan to identify both conventional security vulnerabilities and issues specific to AI-generated code, presenting a consolidated list of findings to the user.
Pre-condition	1. The BackBrain system is running and the user is authenticated. 2. The Developer has an active project or workspace loaded within the integrated development environment (IDE). 3. The system has valid access to the project's file structure and dependencies.
Post-condition	1. The scanning process is completed. 2. Identified issues are categorized, prioritized, and displayed in the system's Severity Panel. 3. The system's internal state is updated to reflect the new scan results, enabling subsequent actions like explanation or autofix.

Table 4: UC-01 scan code

Basic Course of Events:

1. The Developer opens a source code file within the IDE or selects a target directory for analysis.
2. The Developer activates the scan, either by using a dedicated UI command or by triggering a pre-configured automatic scan upon file save.
3. The system receives the scan command and initializes its analysis engines.
4. The system concurrently executes two scanning modules:
 - ❖ Static Security Scan: Runs the integrated Semgrep and Gitleaks engine with configured rule sets including OWASP Top 10 and custom rules against the codebase.
 - ❖ Vibe-Code Analysis: Executes specialized AI-model detectors to identify patterns indicative of hallucinations, such as logical inconsistencies, missing imports, and API mismatches.

5. The system aggregates, deduplicates, and prioritizes the findings from both scan modules.
6. The system updates the Severity Panel in the user interface, displaying a list of issues. Each entry includes a brief description, a severity level (e.g., High, Medium, Low), and the file location.
7. The use case concludes with the Severity Panel populated and ready for user interaction.

UC-02: Fix Issue

Attribute	Description
ID	UC-02
Name	Fix Issue
Actor	Developer
Description	This use case describes the process by which a Developer commands the system to automatically apply a corrective patch for a specific, identified vulnerability or logic flaw. The system modifies the source code to resolve the issue while ensuring syntactic validity.
Pre-condition	1. The Scan Code (UC-01) use case has been successfully executed. 2. One or more issues with a "Fix Available" status are displayed in the Severity Panel. 3. The target source file is writable and under version control (if applicable).
Post-condition	1. The source code in the target file is modified according to the system's remediation algorithm. 2. The applied fix is logged in the system's history. 3. The status of the specific issue in the Severity Panel is updated to "Fixed." 4. The Developer is notified of the successful application.

Table 5: UC-02 fix issue

Basic Course of Events:

1. The Developer reviews the list of issues in the Severity Panel following a scan.
2. For an issue tagged with a "Fix Available" indicator, the Developer selects the accompanying "Apply Fix" button.
3. The system validates that the fix is still applicable to the current state of the code.
4. The system generates a precise diff patch and applies the changes directly to the source file.
5. The system performs an immediate, lightweight syntax verification on the modified file to ensure it remains compilable.
6. Upon successful verification, the system updates the issue's status to "Fixed" in the Severity Panel and provides a visual confirmation (e.g., a checkmark) next to the issue.
7. A non-intrusive notification confirms the fix has been applied successfully.

UC-03: Revert Fix

Attribute	Description
ID	UC-03
Name	Revert Fix
Actor	Developer
Description	This use case allows a Developer to instantly undo a previously applied automatic fix, restoring the affected code segment to its exact original state. This ensures developer control and provides a safety mechanism for any undesired automated changes.
Pre-condition	1. The Fix Issue (UC-02) use case has been successfully executed on a specific issue. 2. The issue's current status in the Severity Panel is "Fixed." 3. The system has retained the pre-fix version of the code in its local history.
Post-condition	1. The modified code is reverted to its state prior to the autofix application. 2. The issue's status is updated from "Fixed" back to "Detected" in the Severity Panel. 3. The revert action is logged in the system's history.

Table 6: UC-03 revert fix

Basic Course of Events:

1. The Developer locates a "Fixed" issue within the Severity Panel.
2. The Developer selects the "Revert Fix" option associated with that issue.
3. The system retrieves the original code content from its local history cache.
4. The system writes the original content back to the source file, effectively undoing the patch.
5. The system updates the issue's status in the Severity Panel from "Fixed" back to "Detected."
6. A confirmation message is displayed to notify the Developer that the revert operation is complete.

UC-04: View Graph

Attribute	Description
ID	UC-04
Name	View Graph
Actor	Developer
Description	This use case enables the Developer to visualize the architecture of the active codebase as an interactive, navigable graph. This visualization maps files and modules as nodes and their dependencies as edges, facilitating architectural understanding and the identification of structural complexity or weakness.
Pre-condition	1. A project workspace is successfully loaded within the BackBrain-integrated environment. 2. The system has permission to parse the project's file structure.
Post-condition	1. The Graph View panel is rendered and displayed in the user interface. 2. An interactive node-and-edge diagram representing the codebase architecture is available for exploration.

Table 7: UC-04 view graph

Basic Course of Events:

1. The Developer navigates to and activates the "BackBrain Graph" view from the main application menu or sidebar.
2. The system initiates a parsing routine on the current workspace to identify files, modules, and their import/export relationships.
3. The system constructs an internal graph model from the parsed data.
4. The system renders this model visually within the Graph View panel. Files/folders are represented as nodes, and dependencies are shown as connecting edges.
5. The Developer can interact with the graph by clicking nodes to highlight connections, zooming, panning, or collapsing/expanding sections of the architecture.

UC-05: Generate Report

Attribute	Description
ID	UC-05
Name	Generate Report
Actor	Developer
Description	This use case allows the Developer to compile a formal, exportable document summarizing the current state of the codebase based on the latest scan results. The report includes identified issues, applied fixes, and overall metrics, formatted for compliance audits or stakeholder review.
Pre-condition	1. The Scan Code (UC-01) use case has been executed at least once in the current session. 2. There is sufficient disk space available in the user's default download or selected directory.
Post-condition	1. A report document (PDF/HTML) is created and saved to the specified file location. 2. The report accurately reflects the aggregated scan data, issue list, and remediation history up to the point of generation.

Table 8: UC-05 Generate report

Basic Course of Events:

1. The Developer selects the "Generate Report" function from the application's tools or export menu.
2. The system presents a dialog for the Developer to select the desired report format (PDF or HTML) and a save location.
3. The Developer confirms the selections.
4. The system aggregates all data from the most recent scan, including issue details (type, severity, location), fixed issues, and project metadata.
5. The system formats this data according to a professional template for the chosen format.
6. The system writes the finalized report to the specified file path on disk.
7. Upon successful creation, the system confirms completion and may offer to open the generated report.

UC-06: Explain Issue

Attribute	Description
ID	UC-06
Name	Explain Issue
Actor	Developer
Description	This use case describes the process where a Developer requests a detailed, contextual explanation for a specific detected issue. The system leverages an external AI service to generate a natural-language analysis that clarifies the problem's nature, potential impact, and the reasoning behind the suggested fix, serving an educational purpose.
Pre-condition	1. An issue is currently selected or highlighted within the Severity Panel. 2. The system has a valid configuration and network connection to communicate with the secondary AI Provider actor. 3. The selected issue contains sufficient contextual code for analysis.
Post-condition	1. A detailed explanation of the selected issue is retrieved and displayed within the user interface. 2. The explanation is cached locally to optimize performance for repeated views of the same issue.

Table 8: UC-06 Explain issue

Basic Course of Events:

1. The Developer selects a specific issue from the list in the Severity Panel.
2. The Developer clicks the "Explain" button or link associated with the selected issue.
3. The system packages relevant context, including the code snippet, error type, location, and any available fix details, into a request payload.
4. The system sends this request to the configured external AI Provider service via a secure API call.
5. The AI Provider processes the request and returns a plain-text, markdown-formatted explanation.
6. The system receives the response and displays the explanation in a dedicated panel (e.g., an "Explanation" tab next to the Severity Panel).
7. The Developer reads the explanation to understand the issue's root cause and implications.

2.3.3 Business Rule Documentation

Identifier	Name	Description	Reference	Revision History
BR-01	Severity Classification	All security and code quality issues identified by the system must be categorized according to a standardized severity scale. Issues shall be classified as Critical, High, Medium, Low, or Informational. This classification must be algorithmically determined based on a combination of Common Vulnerability Scoring System (CVSS) metrics and an assessment of the potential impact on system integrity, data security, and operational functionality.	OWASP Risk Rating Methodology, CVSS v3.1 Specification	v1.0
BR-02	Safe Autofix	Any automatic code modification applied by the system must preserve the syntactic validity of the codebase. The primary rule is that an autofix must not break the build compilation. Immediately after applying a fix, the system is required to perform a syntax verification. If this verification fails, the system must automatically roll back the change to the previous state, log the failure, and notify the user with a descriptive error message.	System Architecture Document	v1.0
BR-03	Privacy & Offline Mode	User source code is considered sensitive data. The system shall not transmit any code to external AI providers without obtaining explicit, prior authorization from the user, typically via an opt-in setting. Core security scanning functionalities, primarily those powered by the local Semgrep engine, must be designed to operate entirely offline without requiring a network connection.	Project Privacy Policy, Software Requirements Specification (SRS)	v1.0
BR-04	Revert Window	To support the Revert Fix (UC-03) use case, the system must maintain a secure, session-based history of all automatically applied fixes. This history must retain the original, pre-fix code content for each modified file to allow for immediate and accurate reversion. The history may be purged at the end of the user's session or upon closing the project workspace.	Use Case UC-03	v1.0

Table 9: Business Rule Documentations

2.3.4 User Interface prototype

Its user interface is composed of three primary, integrated components that work in concert to provide a unified workflow for security analysis, architectural understanding, and code remediation. The prototype emphasizes contextual awareness, minimizing disruption to the developer's primary task of writing code.

1. Severity Panel

The Severity Panel is a dedicated sidebar or pane that provides a centralized dashboard for all code quality and security findings. It is the primary interface for triaging and acting upon issues detected by the system.

- ❖ **Layout & Organization:** Issues are dynamically grouped and sorted by their severity level (Critical, High, Medium, Low, Info) as defined by business rule BR-01. Each entry clearly displays the issue type, a brief description, the affected file name, and line number.
- ❖ **Interactive Actions:** For every listed issue, contextual action buttons are present. A "Fix" button is enabled when an automated correction is available, initiating use case UC-02. An "Explain" button triggers use case UC-06, fetching a detailed AI-generated rationale. A "Revert" button appears for issues previously fixed, enabling use case UC-03.
- ❖ **Functionality:** The panel includes filtering options (by severity, file, or type) and search capabilities. It serves as the launch point for generating compliance reports (UC-05) and provides a real-time summary of the project's security posture.

2. Node Canvas (Graph View)

The Node Canvas is an interactive visualization tool that renders the codebase as a dynamic, navigable graph, fulfilling functional requirement REQ-05. It translates abstract code structure into an intuitive visual model.

- ❖ **Visual Representation:** Files, modules, or key classes are represented as distinct nodes. Dependencies, imports, exports, and data flows between these entities are depicted as connecting edges, creating a map of architectural relationships.
- ❖ **Interactivity:** Developers can click and drag to navigate the graph, zoom in on specific clusters, or click any node to highlight its direct dependencies. Selecting a node typically opens the corresponding file in the Code Editor, creating a direct link between the architectural and code-level views.
- ❖ **Analytical Overlay:** The visualization can be enhanced with analytical overlays, such as color-coding nodes based on scan results, allowing developers to spot structural weaknesses at a glance.

3. Code Editor Integration

BackBrain deeply integrates with the native IDE code editor to provide in-context, real-time feedback without requiring the developer to shift focus to a separate tool.

- ❖ **Inline Decorations:** The system annotates the source code directly within the editor. Lines containing detected issues are underlined with colored squiggles, corresponding to the severity levels in the Severity Panel.
- ❖ **Hover Providers:** When a developer hovers their cursor over a squiggle-decorated line, a hover tooltip appears. This tooltip provides an immediate summary of the issue, its severity, and offers a "Quick Fix" option to apply the most relevant correction instantly, streamlining the UC-02 workflow.
- ❖ **Contextual Awareness:** All decorations and actions are updated in real-time as the developer types or as scans complete, ensuring the feedback is always relevant to the current state of the code.

Unified Workflow: The interface is designed for a fluid workflow. A developer might notice a squiggle in the Code Editor, hover for details, and apply a quick fix. They can then open the Severity Panel to review a full list of remaining issues or generate a report. To understand a complex dependency, they can switch to the Node Canvas to visualize relationships before making further changes. This integrated approach ensures that security and quality analysis are intrinsic, rather than intrusive, parts of the development process.

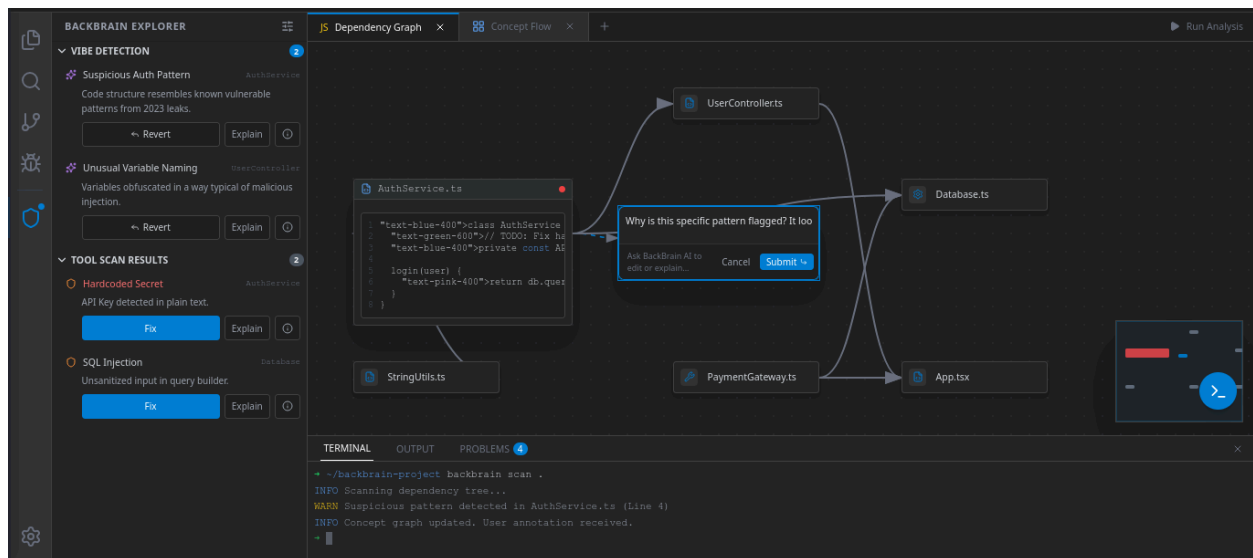


Figure 2: UI Prototype

2.3.5 State chart

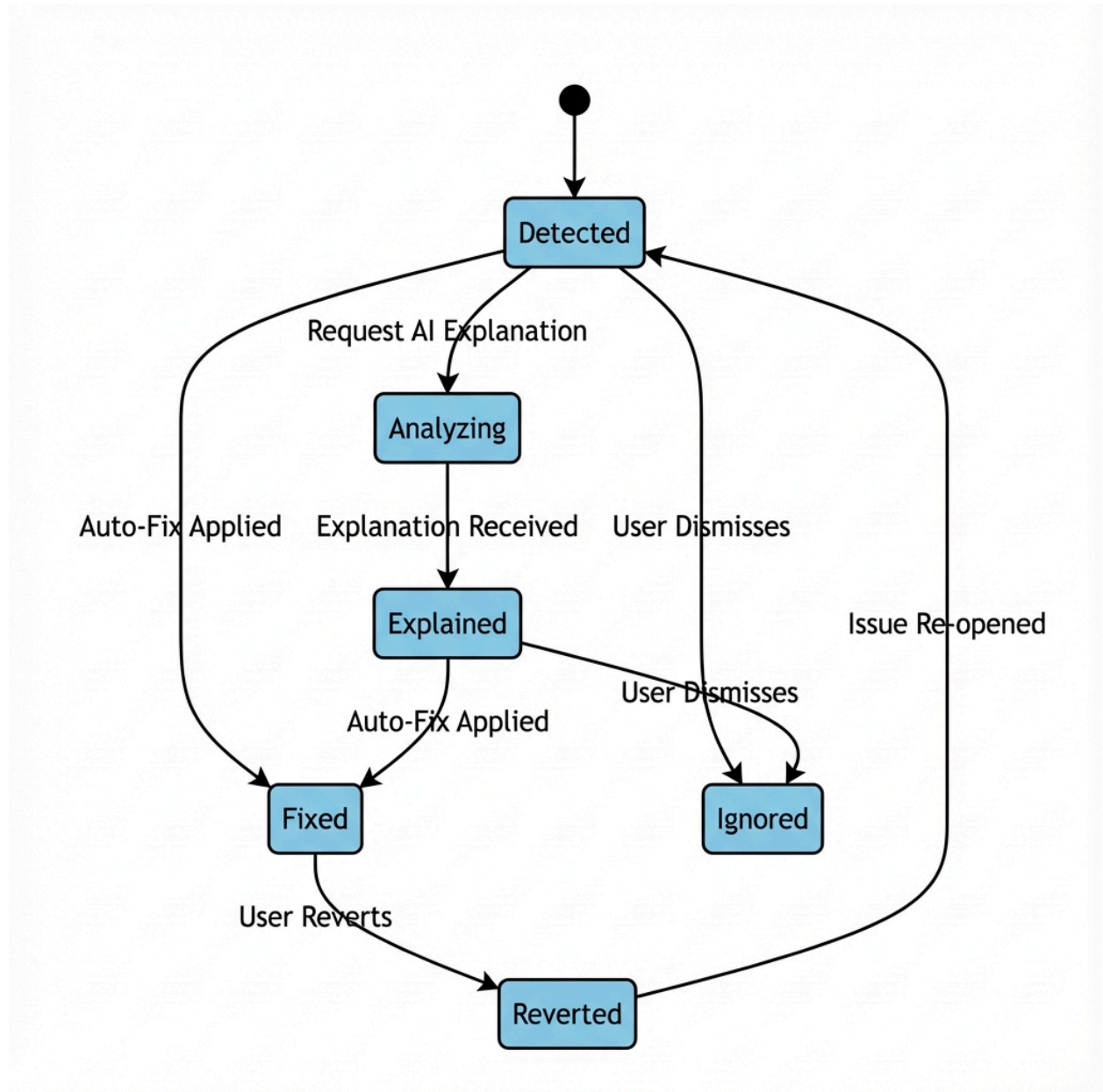


Figure 3: State Chart

The above state chart (Figure 3) formally models the dynamic lifecycle of a CodeIssue object within the BackBrain system. It defines the sequence of states an issue transitions through from detection to final resolution, governed by user actions and system processes. This model ensures a predictable and manageable workflow for handling code vulnerabilities and flaws.

Primary Entity: CodeIssue

Initial State: Detected

State Definitions

State	Description
Detected	The initial and primary active state. The issue has been identified by the scanner and is present in the current code. It is listed in the Severity Panel and visible as an annotation in the Code Editor.
Fixed	The issue has been resolved through the system's autofix functionality (UC-02). The corrective code patch has been applied and verified, and the issue is marked as resolved in the UI. The original code state is preserved in the session history.
Reverted	A transient state entered when a user initiates a revert action (UC-03) on a Fixed issue. This state immediately triggers an automatic transition back to Detected, as the original problematic code is restored.
Ignored	A user-driven state indicating that the Developer has consciously dismissed the issue. The issue is removed from the active Severity Panel list but may be restored through a user preference setting.
Analyzing	A temporary processing state entered when the user requests an AI explanation (UC-06). The system is querying the external AI Provider. Upon receiving the explanation, the issue returns to the Detected state, now augmented with contextual information for the user.

Table 10: state definitions

Transition Rules

The transitions between states are triggered by specific user actions or system events, as detailed below:

- ❖ Detected → Fixed
 - Trigger: User selects the "Apply Fix" action (or "Quick Fix" from hover).
 - Action: System validates and applies the autofix, verifies syntax, and updates the UI.
- ❖ Fixed → Reverted
 - Trigger: User selects the "Revert Fix" action on a fixed issue.
 - Action: System retrieves the original code from session history and restores the file.

- ❖ Reverted → Detected
 - Trigger: Automatic, immediate follow-on from the revert action.
 - Action: System updates the issue status back to Detected in the UI and logs the revert.
- ❖ Detected → Ignored
 - Trigger: User explicitly dismisses or ignores the issue via the UI.
 - Action: Issue is removed from the active Severity Panel view and logged as user-ignored.
- ❖ Ignored → Detected
 - Trigger: User changes a filter setting to "Show Ignored Issues" and chooses to restore it.
 - Action: Issue is returned to the active Detected list in the Severity Panel.
- ❖ Detected → Analyzing
 - Trigger: User selects the "Explain" action on an issue.
 - Action: System packages issue context and dispatches a request to the AI Provider.
- ❖ Analyzing → Detected
 - Trigger: System successfully receives a response from the AI Provider.
 - Action: Explanation is cached and displayed to the user; the issue's state is refreshed as Detected.

The state chart illustrates a controlled lifecycle where an issue is primarily managed between Detected and Fixed states. The Reverted state ensures a clean audit trail for undo operations, while the Analyzing state manages the asynchronous process of fetching expert explanations. The Ignored state provides necessary user control over issue prioritization without permanent deletion.

This model is fundamental to the system's internal logic and directly supports the workflows described in the functional requirements and use cases, particularly REQ-3, REQ-4, REQ-6, UC-02, UC-03, and UC-06.

2.3.6 Activity Diagram

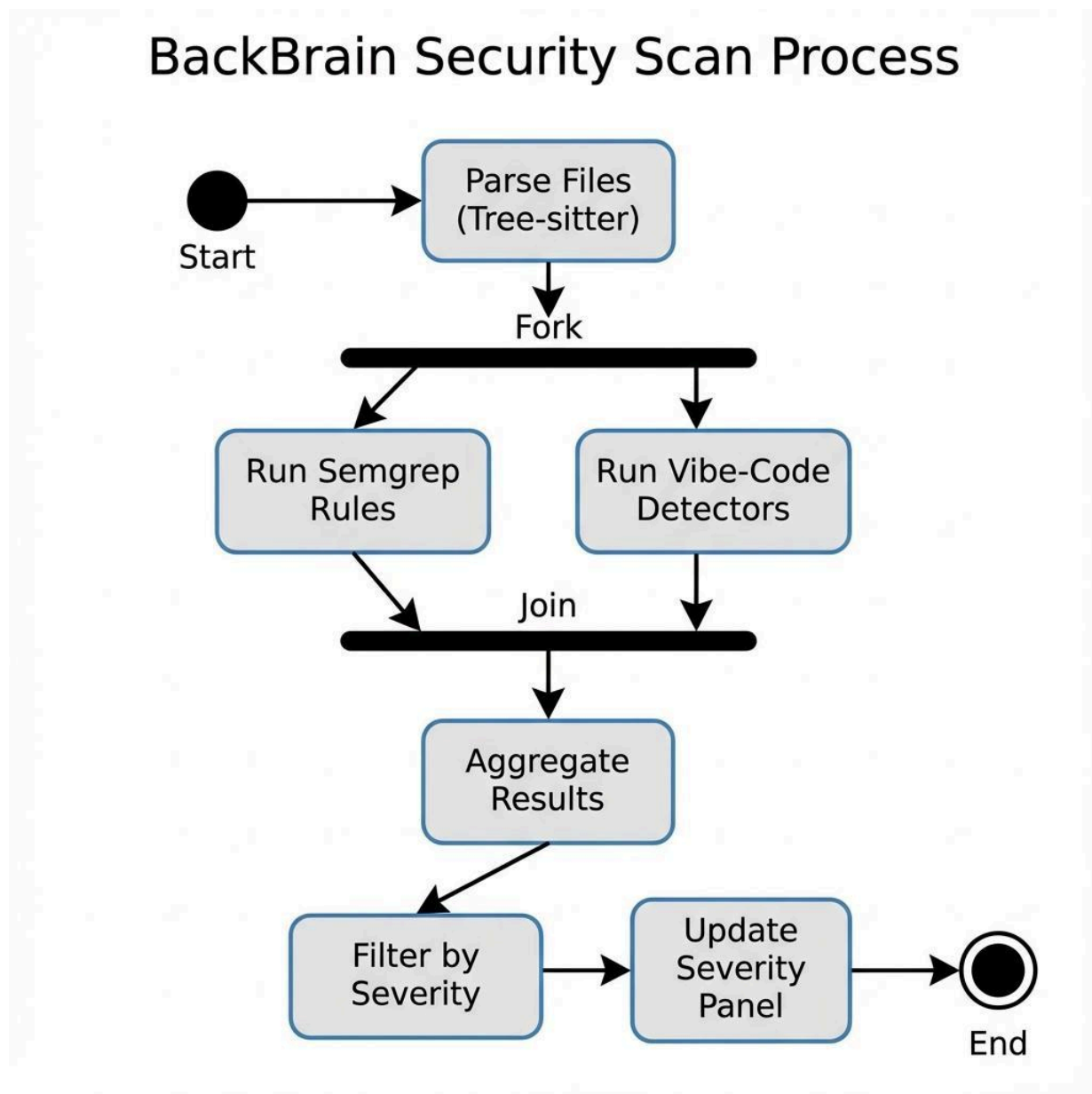


Figure 4: Activity Diagram

The above activity diagram (Figure 4) models the end-to-end process for the Scan Code (UC-01) use case. This workflow is engineered for high performance and comprehensive coverage, executing dual scanning methodologies in parallel to identify both conventional security vulnerabilities and AI-specific anomalies within a codebase.

Process Description

The security scan process commences when a Developer initiates a scan, either manually or via an automated trigger. The system's first action is to parse all relevant files within the active workspace. This parsing utilizes the Tree-sitter library to generate a precise Abstract Syntax Tree (AST) for each file, creating a structured, machine-readable representation of the code's syntax and grammar.

Following the initial parsing, the process splits into two parallel, independent processing lanes to maximize efficiency:

1. **Static Security Analysis Lane:** In this lane, the system executes a battery of Semgrep rules. This engine scans the AST against a curated set of patterns designed to detect known security vulnerabilities (e.g., OWASP Top 10), common bugs, and exposed secrets (like API keys or passwords). The output is a list of raw security findings.
2. **Vibe-Code Analysis Lane:** Concurrently, the system runs specialized Vibe-Code detectors. These are custom algorithms and AI models that analyze the AST and code context to identify patterns indicative of AI hallucinations. This includes detecting logical inconsistencies, name mismatches, calls to deprecated or non-existent APIs, and syntactically valid but semantically flawed code constructs.

Once both parallel analysis streams are complete, their results are synchronized and merged. The system aggregates the findings, performing deduplication and correlation where the same code flaw may be flagged by both engines. This creates a consolidated list of all identified issues.

Subsequently, this master list is filtered against the user's or project's configured severity thresholds (as defined in business rule BR-01). Only issues meeting or exceeding the minimum severity level are retained for presentation.

The process culminates by updating the user interface. The filtered and prioritized list of issues is dispatched to the Severity Panel, where they are displayed for developer review. Simultaneously, inline annotations (squiggles) are rendered within the Code Editor to provide contextual visual cues. This final step satisfies the post-condition of the Scan Code use case, presenting actionable intelligence to the developer.

The activity diagram illustrates a sophisticated, parallel processing pipeline that begins with code parsing and diverges into specialized analysis paths. These paths converge to synthesize a unified set of findings, which are then refined and delivered to the developer interface. This design ensures a thorough and efficient scanning process that is central to the BackBrain system's proactive security model.

2.3.7 Sequence Diagram

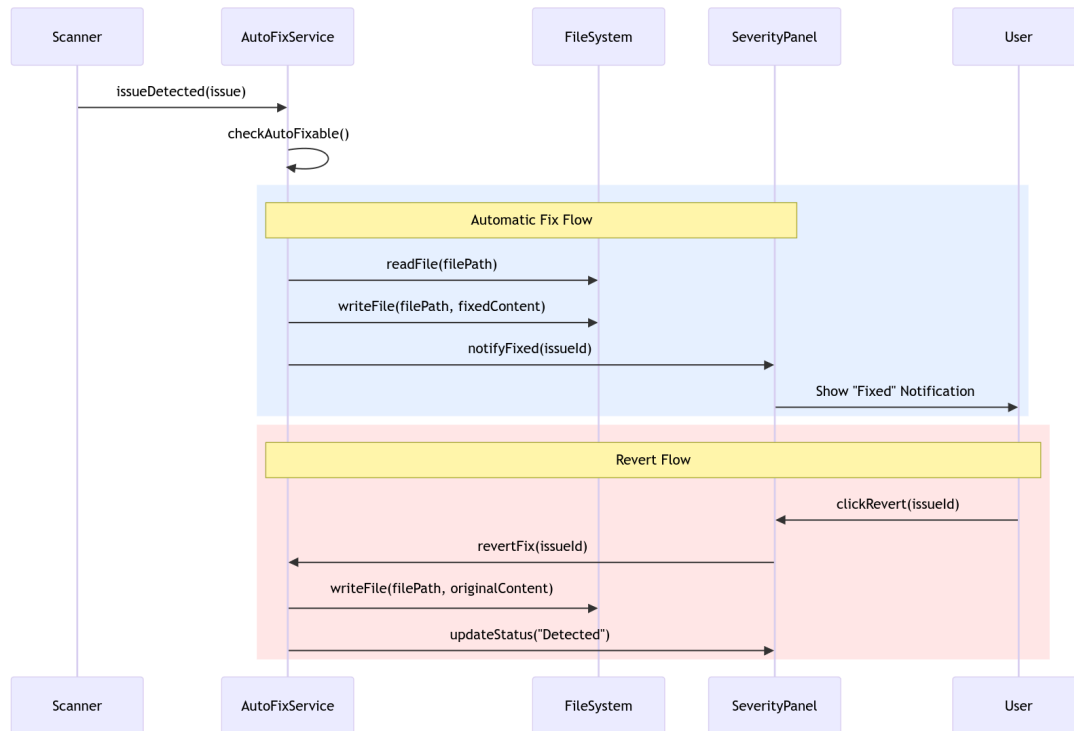


Figure 5: Sequence Diagram

The following sequence diagram (Figure 5) details the collaborative interaction between system components and the user to execute the automated remediation and potential rollback of a code issue. It captures two critical workflows: the primary "happy path" of an automatic fix and the secondary user-initiated revert path. This model demonstrates the system's proactive correction capability and its commitment to maintaining ultimate developer control.

Process Description

This diagram outlines the message flow beginning when a security or code quality issue is first identified. The process involves four primary participants: the Developer, the Severity Panel, the AutoFixService, and the File System.

Primary Sequence (Autofix Path):

The sequence is initiated when the Scanner component detects a new issue and notifies the Severity Panel. The panel updates its display and signals the AutoFixService to evaluate the new issue. The service performs a critical check against its rule database to determine if the specific issue type is eligible for automatic correction.

If the issue is auto-fixable, the service proceeds without requiring manual initiation, aligning

with the system's proactive design principle. It first reads the original content of the affected source file from the File System. Using its patching algorithms, the service generates a corrected version of the code. It then writes this new content back to the file. Following business rule BR-02 (Safe Autofix), the service performs an immediate syntax validation. Upon successful verification, it sends a confirmation notification back to the Severity Panel, which updates the issue's status to "Fixed" and may display a transient success message to the Developer. This entire flow is designed to be rapid and minimally intrusive.

Secondary Sequence (Revert Path):

The revert sequence represents the user's ability to override an automated decision. It begins when the Developer, upon reviewing an applied fix, clicks the "Revert" button in the Severity Panel. This action sends a `revertFix(issueId)` command to the `AutoFixService`.

The service then retrieves the original pre-fix content of the file from the session history, which is maintained according to business rule BR-04 (Revert Window). It writes this original content back to the File System, effectively undoing the change. Finally, it notifies the Severity Panel to update the issue's state from "Fixed" back to "Detected," completing the cycle and returning the developer to a state of informed awareness.

This sequence diagram elucidates the closed-loop automation and control inherent in the BackBrain system. The primary sequence demonstrates an efficient, automated remediation pipeline that minimizes context-switching for the developer. The secondary sequence underscores a fundamental design tenet: that automation must be transparent and reversible. Together, these sequences operationalize the use cases UC-02 (Fix Issue) and UC-03 (Revert Fix), ensuring the system is both powerful and trustworthy.

2.3.8 Analysis Class Model

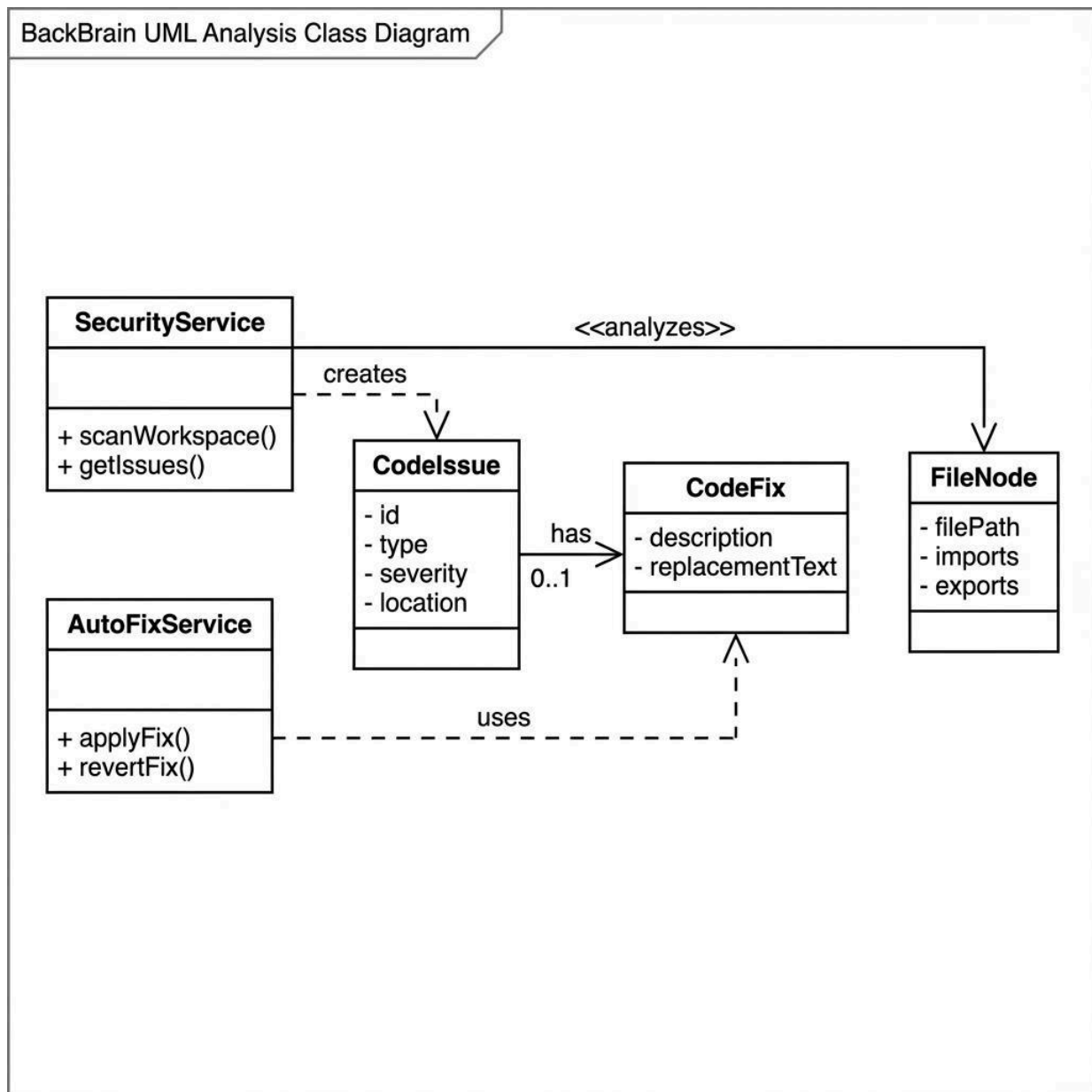


Figure 6: Analysis Class Model

The Analysis Class Model presented in Figure 6 establishes the core conceptual entities, their attributes, and their collaborative relationships within the BackBrain system. This model provides a structural blueprint that directly supports the functional requirements and use case behaviors detailed in previous sections.

The diagram identifies five central classes:

1. SecurityService

This class acts as the primary controller for all scanning and analysis operations.

- ❖ Attributes & Methods: It is responsible for orchestrating the scanWorkspace() process and providing the aggregated results via getIssues().
- ❖ Relationships: The service creates instances of CodeIssue based on its analysis. It analyzes FileNode objects to perform its detection logic, as outlined in the *Security Scan Activity Diagram*.

2. FileNode

This entity represents a source code file within the developer's workspace, serving as a fundamental unit for both analysis and visualization.

- ❖ Attributes: Key properties include filePath, imports, and exports.
- ❖ Relationships: It is the target of analysis by the SecurityService. Furthermore, the collection of interconnected FileNode objects forms the basis of the architectural graph visualized in the system's Node Canvas (REQ-05, UC-04).

3. CodeIssue

This is the central entity encapsulating any vulnerability, bug, or code flaw detected by the system.

- ❖ Attributes: It is characterized by a unique id, a type (security flaw, vibe-code hallucination), a severity level (per BR-01), and a location (file and line number).
- ❖ Relationships: A CodeIssue may optionally have (0..1 association) a CodeFix. This relationship reflects the business logic where not all detected issues have an immediately available automated correction.

4. CodeFix

This entity contains the executable solution for a specific CodeIssue.

- ❖ Attributes: It holds a description of the change and the concrete replacementText or patch required to resolve the issue.
- ❖ Relationships: It exists in a one-to-one relationship with a CodeIssue, but only when an automatic remediation is possible.

5. AutoFixService

This controller class manages the automated remediation and rollback lifecycle.

- ❖ Methods: Its core operations are applyFix() and revertFix(), which directly implement the workflows described in UC-02 and UC-03.
- ❖ Relationships: The service uses the CodeFix object to execute corrections. It interacts with CodeIssue objects to change their state and leverages the FileNode to read from and write to the file system.

The collaborative workflow defined by this model begins with the SecurityService analyzing FileNode objects to instantiate CodeIssue entities. When a CodeIssue has an associated CodeFix, the AutoFixService can execute the fix, modifying the underlying file. This clear separation of concerns between analysis, issue representation, solution definition, and solution execution ensures a modular, maintainable, and scalable architecture that fulfills the system's core mission of integrated, actionable code analysis.

2.3.9 Logic model

The following logic model presents the core algorithms that operationalize the BackBrain system's primary functionalities. These algorithms define the procedural steps, decision points, and data transformations that underpin key use cases, including workspace scanning, vibe-code detection, and automated remediation.

Algorithm 1: Security Scan Orchestration

This algorithm implements the workflow for the Scan Code (UC-01) use case. It orchestrates a comprehensive, parallel analysis of a workspace by coordinating multiple scanning engines to maximize coverage and efficiency. The design prioritizes performance through concurrent execution while ensuring results are aggregated and filtered for clarity.

```
FUNCTION ScanWorkspace(workspacePath) :  
  
    files = FileSystem.GetFiles(workspacePath)  
  
    astMap = TreeSitter.parseAll(files) issues = [] // Parallel Execution  
    semgrepResults = SemgrepScanner.scan(files)  
  
    vibeResults = VibeCodeScanner.scan(astMap) issues.addAll(semgrepResults)  
    issues.addAll(vibeResults) return FilterAndSort(issues)  
  
END FUNCTION
```

Algorithm 2: Vibe-Code Detection (Name Mismatch)

This algorithm provides a concrete example of the specialized logic within the VibeCodeScanner for identifying a specific class of AI hallucination. It checks for consistency between imported and exported symbols across files, a common error in AI-generated code where fabricated or misnamed APIs are referenced.

```
FUNCTION DetectNameMismatches(astMap) :  
    issues = []
```

```

FOR each file in astMap:
    definedSymbols = ExtractExports(file.ast)
    FOR each import in file.imports:
        targetFile = ResolveImport(import.path)
        IF targetFile exists AND import.name NOT IN targetFile.exports:
            issues.add(NewIssue("Name Mismatch", file, import.line))
        END IF
    END FOR
END FOR
return issues
END FUNCTION

```

Algorithm 3: Autofix Application

This algorithm details the precise steps for the Fix Issue (UC-02) use case, enforcing the safety and reliability mandated by business rule BR-02 (Safe Autofix). It ensures every automated code modification is preceded by a backup and followed by a validation step, with an automatic rollback mechanism in case of failure.

```

FUNCTION ApplyFix(issueId):
    issue = GetIssue(issueId)
    fileContent = FileSystem.read(issue.location.file)

    // Create backup for revert
    SaveBackup(issue.id, fileContent)

    newContent = ReplaceString(fileContent, issue.location.range,
issue.fix.replacement)

    IF SyntaxCheck(newContent) IS VALID:
        FileSystem.write(issue.location.file, newContent)
        return Success
    ELSE:
        return Error("Fix resulted in syntax error")
    END IF
END FUNCTION

```

These algorithms translate the system's functional requirements into executable logic. Algorithm 1 showcases the high-level parallel scan orchestration. Algorithm 2 exemplifies a specific, semantic detection rule for AI-generated code flaws. Algorithm 3 demonstrates the robust, safety-first procedure for automated code correction, which is fundamental to user trust and system reliability.

2.4 Non functional requirement

Non-functional requirements define the operational qualities and constraints of the BackBrain system. These requirements ensure the platform is not only functional but also robust, secure, maintainable, and delivers a seamless user experience that aligns with the demanding environment of modern software development.

Performance

The system must deliver immediate feedback to preserve developer focus and workflow efficiency. Security and code-quality scans for an individual file must complete in less than 100 - to provide real-time, in-editor annotations without perceptible delay. For full workspace operations, performance must scale efficiently, with initial scans of large projects completing within a timeframe that does not disrupt productive work, and subsequent incremental scans being nearly instantaneous.

Reliability & Availability

A core tenet of the system is operational independence. All essential scanning and analysis features, particularly the static security analysis powered by Semgrep and Gitleaks, must function entirely offline without any dependency on external network services. This ensures continuous availability and utility regardless of internet connectivity, making the tool reliable in all development environments.

Security

The system must adhere to the highest standards of data protection. When configured to use external AI services for explanations or advanced fix generation, all sensitive credentials, such as API keys, must be stored securely using the host operating system's trusted credential management service (e.g., Windows Credential Manager, macOS Keychain, Linux Secret Service). User source code must never be transmitted to a third-party service without explicit, informed user consent.

Maintainability

To ensure long-term codebase health and facilitate collaborative development, the system's implementation must follow strict software engineering practices. All code will be written in TypeScript with a consistent and well-documented configuration. Maintainability will be enforced through a minimum of 80% unit test coverage for core modules, comprehensive integration tests for critical workflows, and adherence to a clearly defined module architecture that separates concerns between scanning, fixing, and UI components.

Scalability

The architecture must be designed to handle projects of enterprise scale. The system must efficiently process codebases containing in excess of 10,000 files without causing the user interface to freeze or become unresponsive. This requires optimized algorithms for parsing and analysis, efficient memory management, and the use of asynchronous

operations to keep the UI thread available during intensive computational tasks.

Usability & Developer Experience (UX)

The primary design goal is to integrate security and quality assurance into the developer's existing workflow with minimal friction. The system must prioritize a zero-configuration default experience that works immediately upon installation for common frameworks and languages. Automated actions, such as applying safe fixes, should occur silently in the background where possible. Notifications and alerts must be non-intrusive, contextual, and designed to inform without interrupting the developer's cognitive flow, thereby ensuring that security is built-in rather than being a disruptive gatekeeping step.

2.5 System Requirement

This section defines the minimum and recommended hardware and software specifications necessary to install, run, and effectively utilize the BackBrain system. These requirements are derived from the performance and functionality targets established in the non-functional requirements.

2.5.1 Hardware Requirements

The following hardware specifications are necessary to support the computational demands of real-time code analysis, architectural visualization, and automated remediation processes.

- ❖ Processor (CPU):
 - Minimum: A modern dual-core processor.
 - Recommended: A quad-core processor or better is highly recommended. The enhanced parallel processing capability significantly improves the performance of concurrent scanning operations, reducing latency during whole-workspace analysis as outlined in the performance requirements.
- ❖ Memory (RAM):
 - Minimum: 8 GB of system RAM.
 - Recommended: 16 GB of system RAM. This is advised for optimal performance when working with large codebases (10k+ files) or when generating and manipulating complex dependency graphs, ensuring the system remains responsive and does not cause UI freezing.
- ❖ Storage:
 - Minimum: 500 MB of available disk space for the application installation, local rule sets, and caching of analysis data.
- ❖ Network:
 - A broadband internet connection is required only for optional features that leverage external AI provider services (e.g., in-depth issue explanations). Core security scanning and autofix functionalities are designed to operate fully offline to meet reliability requirements.

2.5.2 Software Requirements

The BackBrain system is designed as an extension for modern integrated development environments and requires the following software platform and tools.

- ❖ Operating System:
 - Windows: Version 10 or 11 (64-bit).
 - macOS: Version 12 (Monterey) or later.
 - Linux: A compatible distribution with glibc support, such as Ubuntu 20.04 LTS or

later, and RHEL/CentOS 8 or later.

- ❖ Development Environment / Editor: The primary deployment target is as an extension for:
 - Visual Studio Code: Version 1.80 or later.
 - Compatible Editors: The system is also designed to be compatible with popular VS Code-compatible editors and forks, including Cursor, Windsurf, Antigravity, VSCodium, and Code-Server. Full functionality is contingent on the host editor's API compatibility with VS Code 1.80+.
- ❖ Runtime & Development:
 - For End Users (VS Code Extension): The required Node.js runtime is bundled with the host editor; no separate installation is needed.
 - For System Development & Contribution: A current JavaScript runtime is required:
 - Node.js: Version 18 or later, or
 - Bun: Version 1.0 or later.
- ❖ Version Control:
 - Git is recommended and may be utilized by the system for enhanced operations, such as providing richer context for code changes or integrating with source-controlled project histories. It is not a strict requirement for core functionality but is considered a best practice in the target development environment.

2.6 Key abstraction with CRC analysis

Class-Responsibility-Collaborator (CRC) analysis is a valuable object-oriented design technique used to identify the core abstractions within a system. By defining the key responsibilities of each class and their necessary collaborations, this model clarifies the distribution of system logic and the interfaces between components. The following CRC cards describe the two central service classes in the BackBrain architecture.

CRC Card: SecurityService

The SecurityService class acts as the primary orchestrator for all code analysis activities. It is responsible for initiating scans, coordinating between different analysis engines, and synthesizing their results into a unified, actionable output for the developer.

Class Name	SecurityService
Responsibilities	Orchestrate Scanning Workflow: Initiate and manage the complete scanning process for individual files or entire workspaces. Aggregate & Synthesize Findings: Collect raw results from disparate scanner engines, merge them, and remove duplicates to create a single, coherent list of issues. Manage Scanner Adapters: Interface with and coordinate the execution of specific scanner plugins (e.g., SemgrepAdapter, VibeCodeAdapter). Prioritize and Filter Issues: Apply business rules and user configurations to rank issues by severity (e.g., prioritizing critical security vulnerabilities over minor style violations). Manage Analysis State: Track the status of ongoing scans and cache results for performance.
Collaborators	ScannerAdapter : Delegates the specialized analysis tasks to the appropriate engine. FileSystem: Reads source code files for analysis. Logger: Records scan events, errors, and performance metrics for debugging and auditing. CodeIssue: Creates and populates instances based on scanner findings.

Table 11: CRC card: SecurityService

CRC Card: AutoFixService

The AutoFixService class is the engine responsible for the automated remediation of code issues. It executes fixes safely, maintains a history for undo operations, and ensures that all automated changes preserve the syntactic integrity of the codebase.

Class Name	AutoFixService
Responsibilities	Apply Code Fixes: Execute the patches defined in a CodeFix object by modifying the content of the target source file in the filesystem. Maintain Fix History: Keep a secure, session-based record of all applied fixes, including the original code state, to enable the revert functionality mandated by BR-04. Verify Fix Safety: Perform a pre-commit validation (e.g., a syntax check) on the modified code to ensure the fix does not break compilation, enforcing BR-02. Manage Reversion: Restore a file to its original state by retrieving the backup from the fix history upon a user's revert request. Coordinate Notifications: Signal the outcome of fix/revert operations to the user interface.
Collaborators	FileSystem: Reads the current file content and writes the fixed or reverted content. SecurityService: Retrieves the CodeIssue and its associated CodeFix object to execute. NotificationService: Informs the UI layer of the operation's success or failure to update the Severity Panel and user notifications. HistoryManager (internal): Manages the cache or data structure storing the backup history of applied fixes.

2.6.1 Conceptual modeling: Class diagram

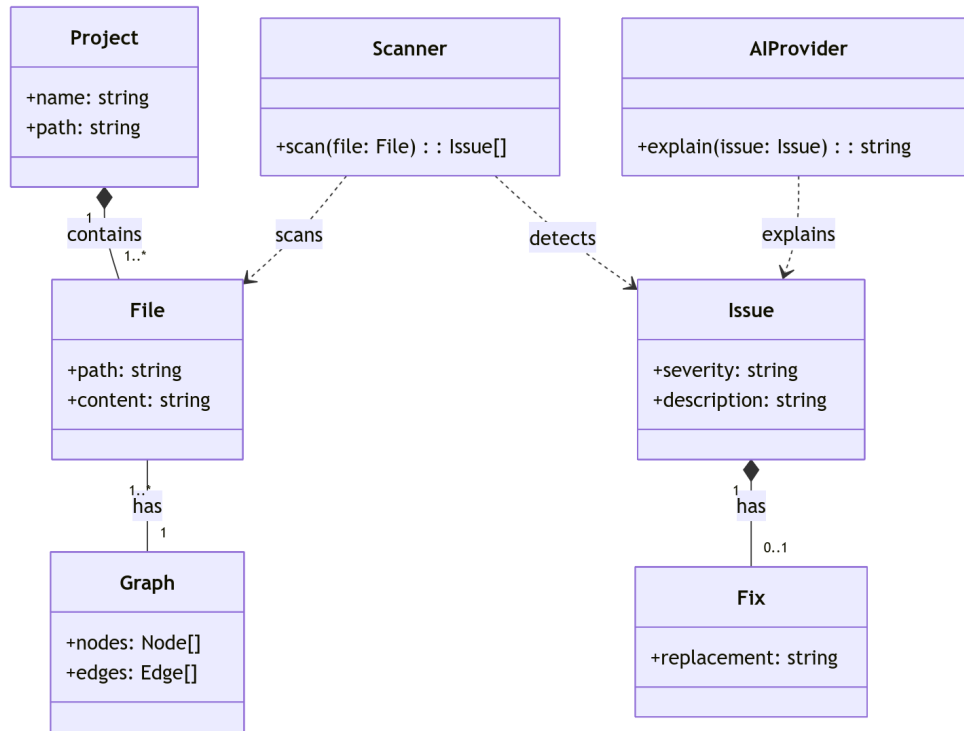


Figure 7: Conceptual Model

The conceptual class diagram in Figure 7 establishes the foundational domain model for the BackBrain system. It visualizes the key abstractions, their primary attributes, and the essential relationships that define the system's core structure and workflow. This high-level model focuses on the "what" rather than the "how," clarifying the problem space before detailed design.

Core Entities and Relationships:

The model centers on the Project, which represents the developer's entire workspace. A Project contains one or more File entities, each defined by its path and content. This aggregation forms the basic unit of analysis.

The collection of Files and their interdependencies are abstracted into a Graph, composed of nodes and edges. This Graph is not a direct container of Files but is a derived visualization that

models the architectural relationships between them, fulfilling the requirement for system comprehension (REQ-05).

The analysis capability is represented by the Scanner class. A Scanner's primary responsibility is to detect problems by analyzing File content, producing one or more Issue instances. Each Issue is characterized by its severity and description. This relationship encapsulates the core Scan Code use case (UC-01).

An Issue may optionally have one proposed Fix, represented by a replacement text patch. This zero-or-one (0..1) cardinality reflects that not all detected issues have an automatic correction available, a key constraint in the system's logic.

Finally, the AI Provider is modeled as an external collaborator. Its responsibility is to explain a given Issue, returning a human-readable string that elucidates the problem's context and rationale. This supports the Explain Issue use case (UC-06).

Model Significance:

This conceptual diagram serves as a bridge between the high-level requirements and the detailed analysis class model. It explicitly shows that:

1. The system operates within the context of a Project and its constituent Files.
2. Analysis (Scanning) and remediation (Fixing) are distinct concepts linked through the Issue entity.
3. Architectural visualization (Graph) and expert explanation (AI Provider) are first-class concerns within the domain.

This clarity ensures that all subsequent design and implementation work remains aligned with the system's fundamental purpose: to proactively detect, explain, and fix code issues within a developer's project.

2.6.2 Identifying change cases

A robust software architecture must not only satisfy current requirements but also anticipate and accommodate future evolution. The following change cases identify probable or planned modifications to the BackBrain system. The architectural design prioritizes modularity and clear interface separation to ensure these changes can be implemented with minimal disruption to the core system.

Priority Shift to Security Scanners

A significant shift in the threat landscape or organizational policy may necessitate prioritizing traditional security vulnerability detection over AI-specific code analysis.

The system is designed with a plugin-based ScannerAdapter interface. Reprioritization

can be managed through configuration changes to scanner execution order and severity weighting within the SecurityService, without requiring code modifications to the analysis pipeline.

Integration of a New AI Provider

The rapid evolution of large language models will necessitate swapping or supplementing the current AI service used for generating issue explanations.

The system defines an abstract AIProvider interface. Integrating a new model (e.g., Llama 3, Claude) requires only the implementation of a new adapter class that conforms to this interface, ensuring the Explain Issue (UC-06) use case remains unchanged.

Addition of a New Security Scanner

Requirements may arise to integrate additional specialized scanning tools, such as Trivy for container images or Snyk for open-source dependencies, to broaden coverage.

The SecurityScanner interface provides a standard contract for all analysis engines. New tools can be incorporated as additional plugins. The SecurityService aggregates findings from all registered scanners, making this an additive change that enhances capability without altering the core scan workflow.

Migration to a Standalone IDE

The system may be ported from a VS Code extension to a standalone desktop application using a framework like Tauri or Electron.

The core application logic (services, scanners, fix engine) is deliberately decoupled from the VS Code API. The current UI layer (SeverityPanel, NodeCanvas) interacts with these core modules through a dedicated interface. A migration would primarily involve replacing the VS Code-specific UI layer with a new implementation for the target framework, leaving the business logic largely intact.

Incorporation of New Detection Methodologies

Advanced analysis techniques, such as dynamic or interactive application security testing (DAST/IAST), may be required to detect a broader range of vulnerabilities.

The SecurityService is designed to orchestrate diverse detection methods. New methodologies can be integrated as specialized components within the service or as new scanner adapters, following the established pattern for extensibility. This ensures the system's analytical capabilities can evolve beyond static and semantic analysis

2.6.3 User Interface Prototyping

The user interface for BackBrain was designed through an iterative prototyping process focused on a core principle: seamless integration. The goal was to embed advanced security and analysis capabilities directly into the developer's existing workflow without introducing disruption or cognitive overload. This section details the high-fidelity prototype that evolved from this process, consisting of three primary, interconnected views.

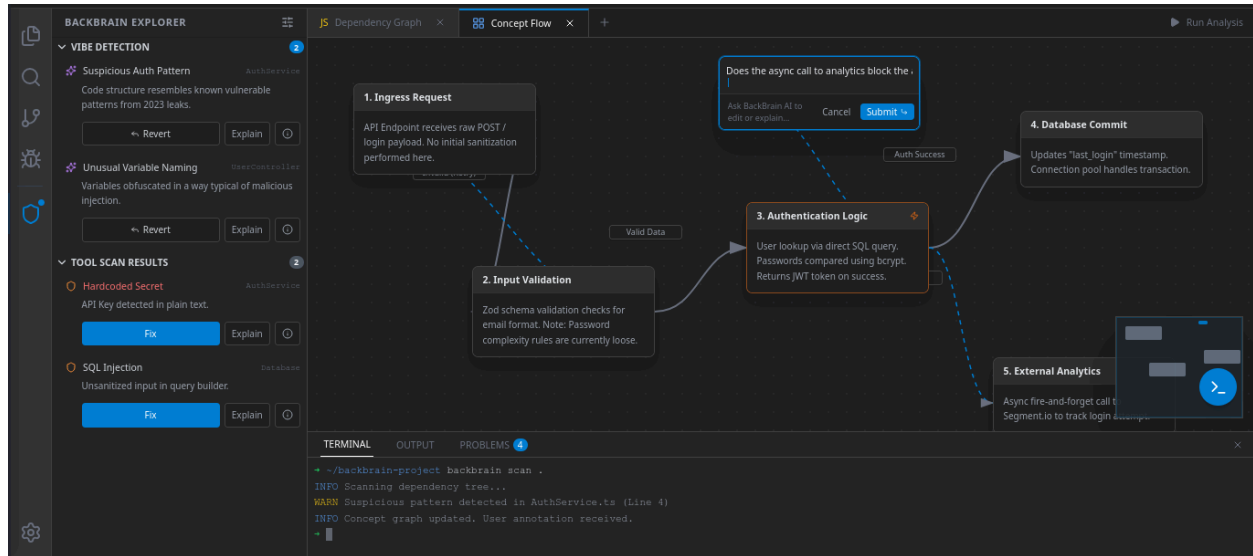


Figure 8: UI Prototype 1

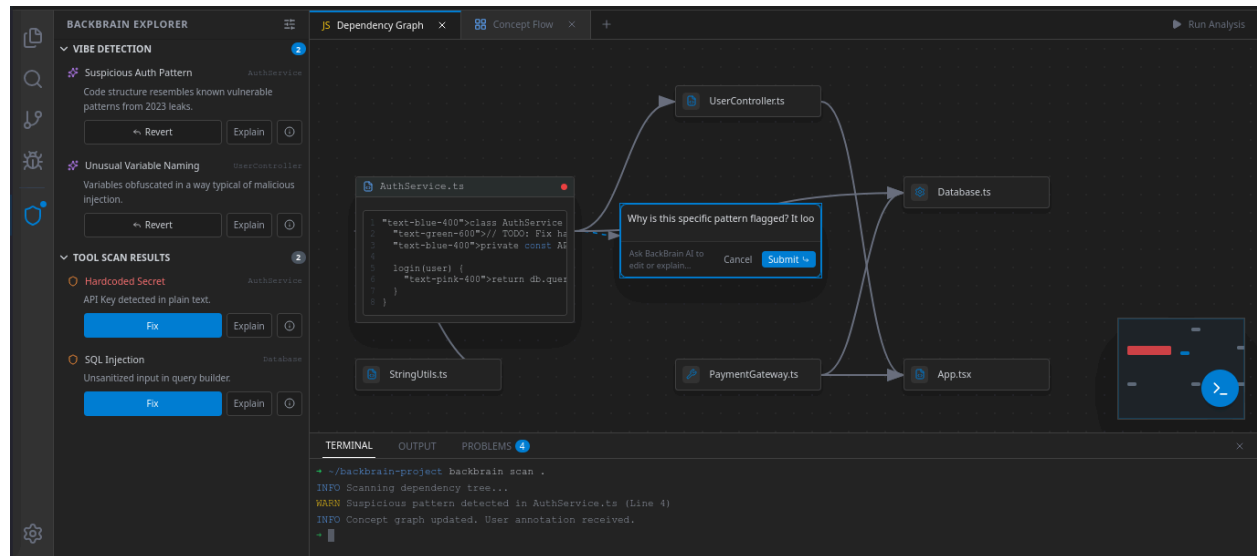


Figure 9: UI Prototype 2

Chapter Three: System Design

This chapter presents the design decisions that shape BackBrain from a high-level architectural perspective down to the individual class definitions and data structures. The design has been crafted with several guiding principles in mind: maintainability, where the codebase must be easy to understand and modify as the security landscape evolves; extensibility, where new scanners, AI providers, and features can be integrated without disrupting the core system; testability, where every component can be verified in isolation and portability, where the core business logic remains independent of any specific runtime environment, enabling the planned migration from a VS Code extension to a standalone integrated development environment.

The design presented herein follows the Object-Oriented Analysis and Design (OOAD) methodology chosen in Chapter One. The Ports and Adapters architectural pattern, also known as Hexagonal Architecture, serves as the foundational structure, ensuring that the application's core remains insulated from external dependencies. This approach directly supports the long-term vision of BackBrain, which involves launching as a VS Code extension for rapid market entry and, upon achieving sufficient user adoption and revenue, evolving into a fully-fledged standalone IDE with integrated AI capabilities.

3.1 Architectural Design

The architectural design of BackBrain reflects a deliberate separation of concerns, where each layer of the system has a distinct responsibility. This separation is critical for a security tool, as it allows the core security analysis logic to be rigorously tested and verified independently of the user interface or the specific file system implementation. The architecture is divided into five logical layers: the Shell Layer, which provides the host environment integration; the User Interface Layer, which handles all visual components and user interactions; the Core Business Logic Layer, which contains the portable, framework-agnostic security analysis engine; the Adapter Layer, which implements the port interfaces for external services; and the External Integrations Layer, which comprises the third-party tools and APIs that the system leverages.

The Shell Layer acts as the thinnest possible integration point with the host environment. In the current implementation, this is the VS Code Extension API, which provides access to the editor, file system, and user interface primitives. The design ensures that the Shell Layer contains no business logic; its sole purpose is to translate host-specific events into calls to the underlying services and to render the results back to the user. When the time comes to migrate to a standalone IDE, this is the only layer that requires a complete replacement. The new shell, built on Tauri or Electron, would implement the same internal contracts, allowing the entire core system to be reused without modification.

The User Interface Layer is built with React and TypeScript, providing a component-based architecture for the visual elements. The primary components include the Severity Panel, which displays a prioritized

list of detected issues; the Diff Preview, which shows proposed code changes before application; the Scan Progress indicator, which provides feedback during analysis; and the Node Canvas, which renders the codebase as an interactive graph. All UI components communicate with the core system through a well-defined message protocol, ensuring a clean separation that facilitates testing and future UI enhancements.

The Core Business Logic Layer is the heart of BackBrain. It is written in pure TypeScript with zero dependencies on any specific UI framework, file system implementation, or AI provider. This layer contains the domain types that model the problem space, such as `CodeIssue`, `CodeFix`, `FileNode`, and `SecurityReport`; the service classes that orchestrate complex operations, such as `SecurityService`, `AutoFixService` and `ScanOrchestrator`; and the security analysis agents that leverage AI for deep code understanding. The independence of this layer is what enables the project's long-term portability goals.

The Adapter Layer implements the port interfaces defined by the core. A port is a TypeScript interface that declares the operations the core logic requires from the outside world, such as reading files, making AI completions, or running security scans. An adapter is a concrete class that implements one of these ports using a specific technology. For example, the `AIProvider` port is implemented by the `VercelAIAdapter`, which wraps the Vercel AI SDK to provide access to multiple large language models. The `SecurityScanner` port is implemented by adapters for Semgrep, Tree-sitter, Gitleaks, and the custom Vibe-Code scanner. This pattern allows the development team to add support for new tools or services by simply creating a new adapter without modifying any core code.

The External Integrations Layer comprises the third-party services and tools that the adapters interact with. This includes the Semgrep static analysis engine for traditional vulnerability detection; Gitleaks for secret and credential scanning; Tree-sitter for high-performance Abstract Syntax Tree parsing; and the various AI provider APIs accessed through the Vercel AI SDK, including OpenAI, Anthropic, Google, xAI, and DeepSeek.

3.1.1 Component Modeling

The component model provides a detailed view of how the system's modules interact with each other, the services they provide, and the services they consume. The diagram below illustrates the major components and their dependencies.

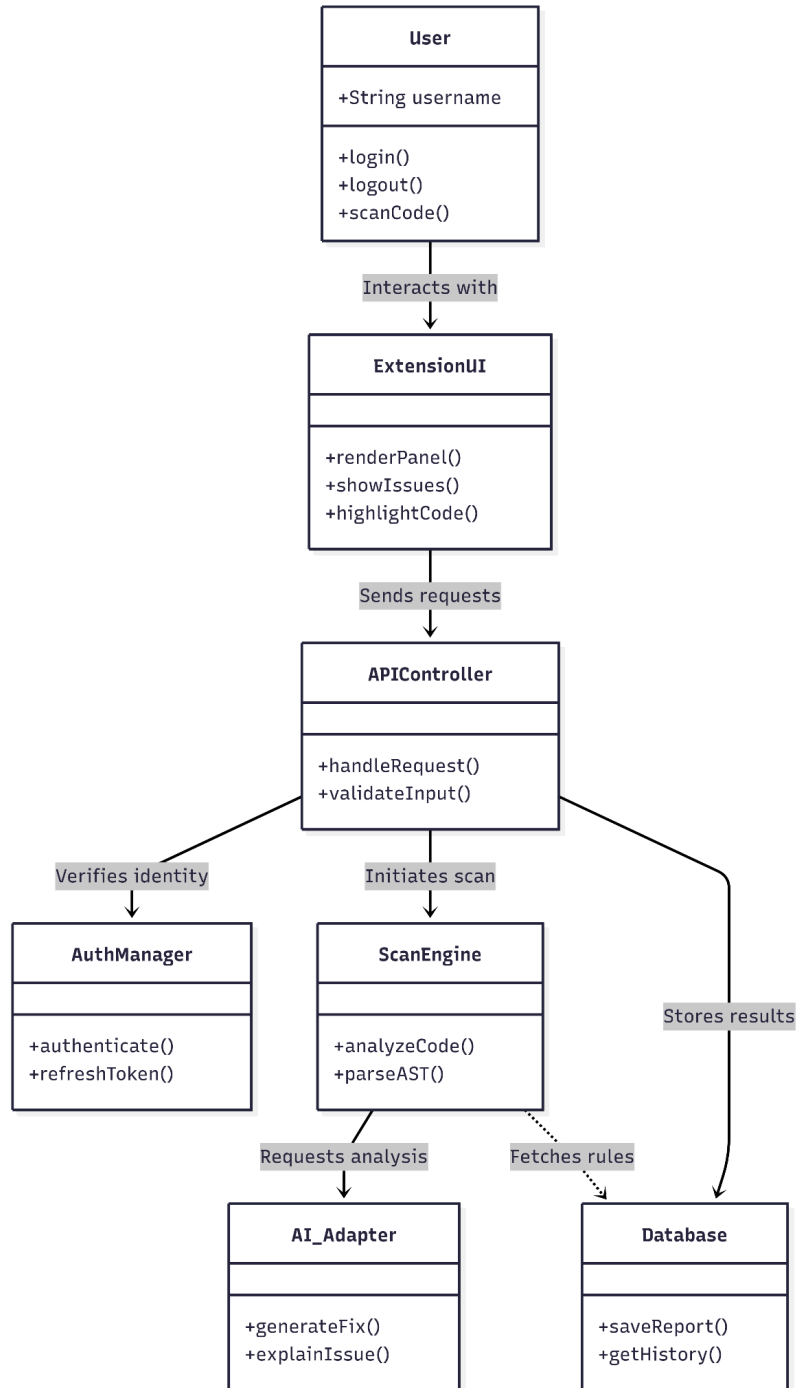


Figure 10: Component Diagram

The component diagram depicts the BackBrain system as a collection of interconnected modules, each with a defined responsibility. The Extension Host Component serves as the entry point when BackBrain runs as a VS Code extension. It is responsible for registering commands, initializing services, and setting up event listeners for file changes and editor actions. This component consumes the SecurityService and ScanOrchestrator services from the core module and provides access to the VS Code Extension API.

The Core Module Component is the central hub of the system. It exports all the domain types, service interfaces, and utility functions required by other components. It is designed to have no dependencies on platform-specific APIs, which is enforced by strict TypeScript configuration. The core module provides the following key services: the ProviderRegistry for managing swappable adapters at runtime; the SecurityService for orchestrating multi-scanner vulnerability detection; the AutoFixService for applying and reverting code modifications; the AIAnalysisService for AI-powered issue explanation and fix generation; and the ScanOrchestrator for coordinating static analysis with AI-agent-based assessment.

The Scanner Adapters Component contains the implementations of the SecurityScanner port. Each adapter wraps a specific scanning technology. The SemgrepScanner adapter invokes the Semgrep CLI and parses its JSON output. The GitleaksScanner adapter detects exposed secrets and credentials. The TreeSitterScanner adapter performs fast AST-based analysis for language-specific patterns. The VibeCodeScanner adapter uses regular expression rules to detect common AI hallucination patterns, such as missing imports or inconsistent naming.

The AI Adapters Component implements the AI provider port using the Vercel AI SDK. The VercelAI Adapter is a unified adapter that supports multiple providers through a consistent interface. It handles the construction of chat messages, the invocation of the generateText and streamText functions from the SDK, and the parsing of responses including token usage tracking. The component is configured using environment variables for API keys, allowing users to supply their own credentials for their preferred provider.

The Security Agents Component contains the AI-powered analysis agents that form the deep analysis tier of the ScanOrchestrator. These agents are distinct from the static scanners; they use large language models to reason about code security in ways that rule-based tools cannot. The ChunkAnalystAgent examines individual code segments for vulnerabilities that require semantic understanding. The ComponentArchitectAgent assesses the security posture of entire modules and their interactions. The FlowAnalysisAgent traces data flows across files to identify cross-cutting vulnerabilities like injection attacks or data leakage. The SentinelAgent synthesizes the findings from all other agents and static scanners, groups related issues, identifies root causes, and generates an executive summary with actionable recommendations.

The Webview UI Component provides the React-based user interface that is rendered within VS Code's webview panel. It consumes the message protocol defined by the SeverityPanelProvider and renders

components for displaying issues, showing fix previews, and tracking scan progress. The component is built using Vite for fast development iteration and optimized production bundles.

The coupling between components is deliberately minimized. The core module has no knowledge of which adapters are being used; it only knows about the port interfaces. The adapters are registered with the ProviderRegistry at startup, and the core services retrieve them by name or type when needed. This loose coupling ensures that changes to one component do not cascade to others, and that new adapters can be introduced without any code changes to the core.

3.1.2 Deployment Modeling

The deployment model describes how the software components are distributed across physical or virtual machines and how they communicate. As a VS Code extension, BackBrain's deployment is primarily client-side, running entirely within the user's development environment. However, certain features, particularly those involving AI-powered analysis, require communication with external services.

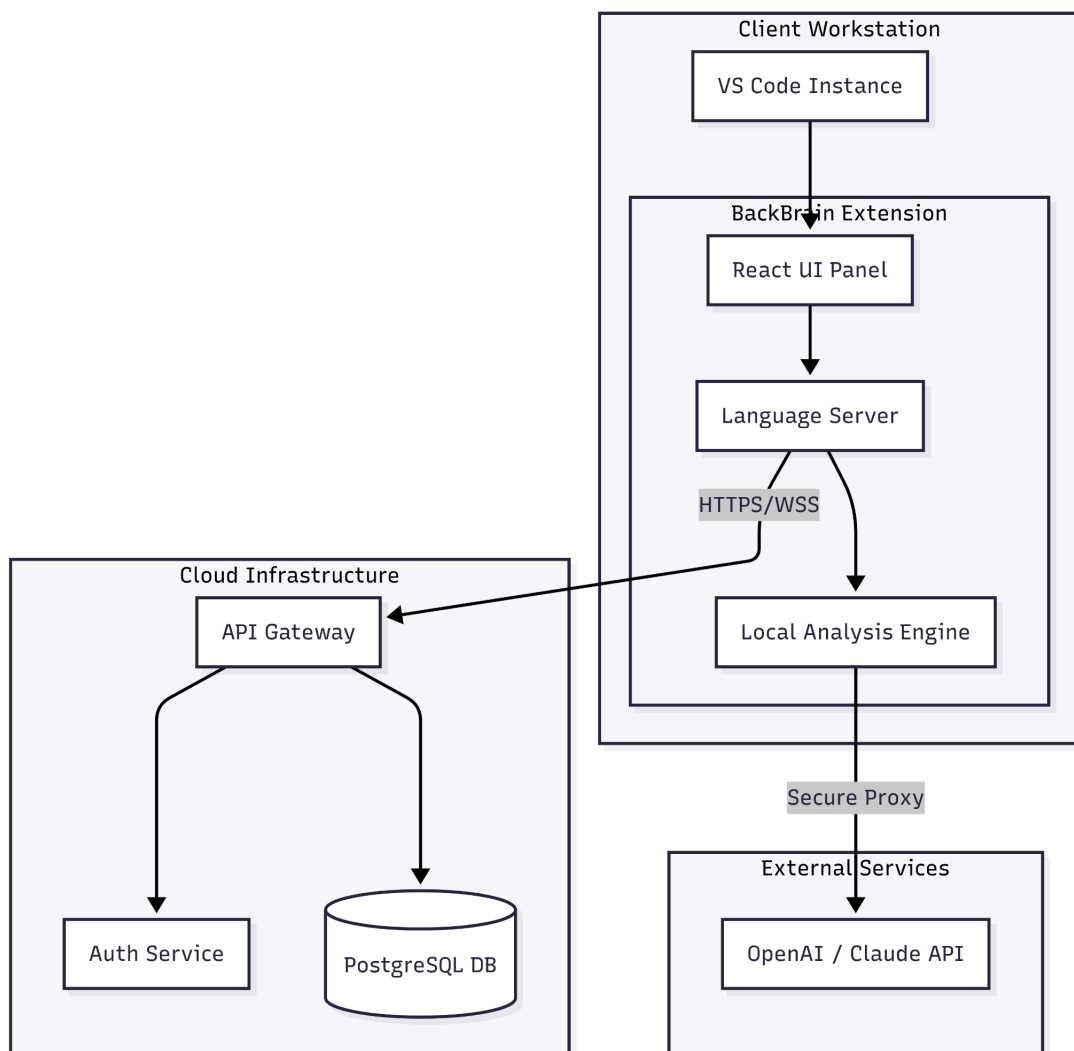


Figure 11: Deployment Diagram

The deployment diagram illustrates the runtime environment for BackBrain. The Developer Workstation node represents the user's local machine where VS Code is installed. Within this node, the VS Code application hosts the BackBrain extension. The extension runs in the Extension Host process, which is a separate Node.js process managed by VS Code. The Extension Host has access to the local file system and can spawn child processes to run command-line tools like Semgrep and Gitleaks.

The Webview Process is a separate Chromium renderer process that hosts the BackBrain user interface. It communicates with the Extension Host through a message-passing API provided by VS Code. This separation ensures that the UI remains responsive even during intensive scanning operations.

The Local File System artifact represents the user's codebase. The extension reads source files for analysis and writes them back when applying autofixes. All file operations go through the FileSystem port, which is implemented by the VSCodeFileSystem adapter. This adapter uses the VS Code workspace API for file access, ensuring proper integration with VS Code's virtual file system capabilities.

The Semgrep CLI and Gitleaks CLI are external command-line tools that are invoked by their respective adapters. These tools run as child processes on the developer's workstation. The extension includes logic to detect whether these tools are installed and, if not, offers to install them automatically. Semgrep performs pattern-based static analysis using a large database of community rules, while Gitleaks scans for hardcoded secrets and API keys.

The AI Provider Cloud Services node represents the external APIs hosted by AI providers such as OpenAI, Anthropic, Google, xAI, and DeepSeek. Communication with these services occurs over HTTPS and requires API keys provided by the user. The VercelAIAdapter handles the construction of requests, including the system prompts and user messages that form the basis of the prompt engineering strategy. All AI interactions are optional; the core security scanning functionality works entirely offline using the local static analysis tools.

The network communication between the extension and the AI provider services uses the standard Node.js https module wrapped by the Vercel AI SDK. The SDK handles retries, error handling, and streaming for long responses. Token usage is tracked for each request, enabling potential future integration with usage-based billing.

The deployment model also accounts for the planned migration to a standalone IDE. In that scenario, the Developer Workstation would run the BackBrain IDE application built on Tauri, which packages a Rust-based backend with a webview frontend. The Rust backend would replace the VS Code Extension Host, implementing the same internal contracts. The webview would continue to use the same React components. The local tool invocations and AI service communications would remain unchanged, demonstrating the portability benefits of the Ports and Adapters architecture.

3.2 Detail Design

The detailed design specifies the internal structure of the system's classes, their attributes, methods, relationships, and data types. This section provides the blueprints that guide implementation, ensuring that the development team has a clear understanding of how each component should be built.

3.2.1 Design Class Model

The design class model expands on the conceptual model from Chapter Two by adding implementation details such as method signatures, attribute types, and access modifiers. The model is divided into three packages corresponding to the major functional areas of the system: the Domain Types package, the Services package, and the Adapters package.

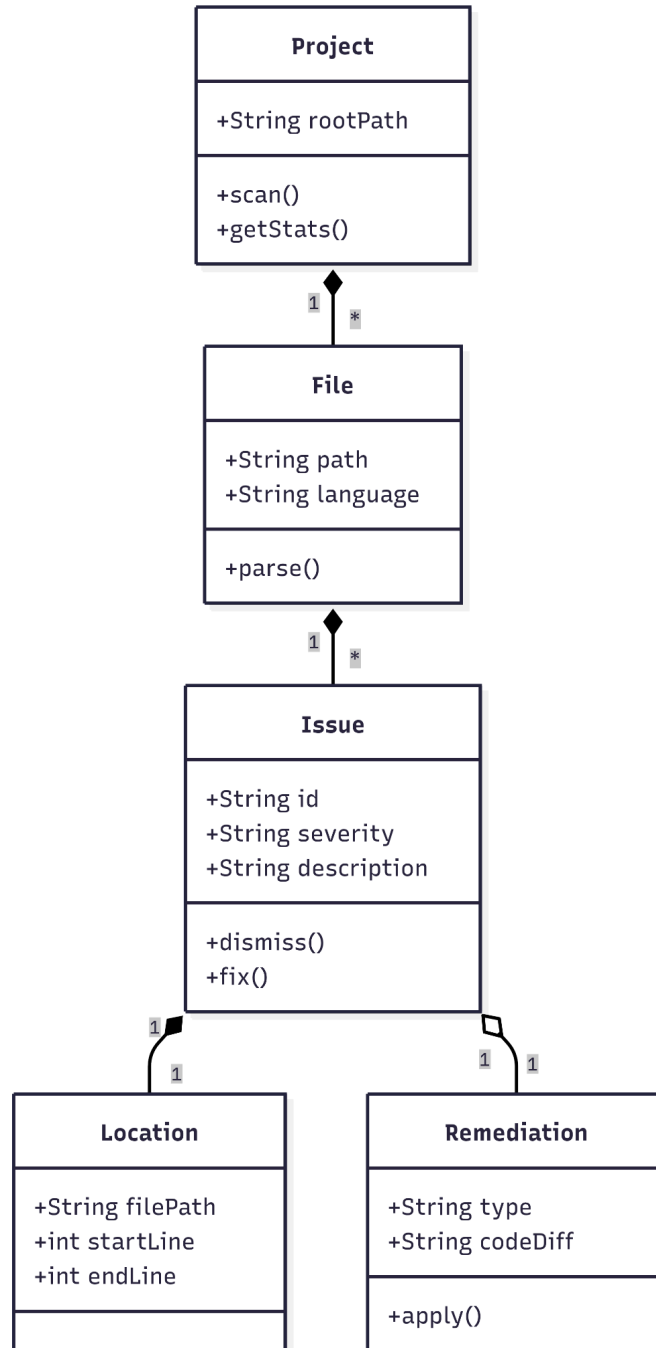


Figure 12: Design Class Diagram

Domain Types Package

The domain types define the data structures that flow through the system. These are implemented as TypeScript interfaces and type aliases, providing strong type safety without the overhead of classes.

The `CodeIssue` interface represents a single detected vulnerability or code quality problem. It has the following attributes: `id` (string), a unique identifier for the issue; `ruleId` (optional string), the identifier of the rule that detected the issue; `type` (`CodeIssueType`), which is a union of literal strings including `'name_mismatch'`, `'flow_gap'`, `'deprecated_api'`, `'missing_dependency'`, `'style_inconsistency'`, `'logic_error'`, `'integration_mismatch'`, `'api_contract_violation'`, and `'security_vulnerability'`; `title` (string), a human-readable summary; `description` (string), a detailed explanation; `location` (`CodeLocation`), specifying the file and position; `severity` (`IssueSeverity`), one of `'critical'`, `'high'`, `'medium'`, `'low'`, or `'info'`; `suggestedFix` (optional `CodeFix`), a proposed remediation; and `category` (`IssueCategory`), one of `'security'`, `'quality'`, `'style'`, `'dependency'`, or `'logic'`.

The `CodeLocation` interface specifies where an issue was found. It includes `filePath` (string), the path to the affected file; `line` (number), the one-indexed line number; `column` (optional number), the column position; `endLine` (optional number), for multi-line issues; and `endColumn` (optional number).

The `CodeFix` interface describes a proposed change. It has `description` (string), explaining what the fix does; `original` (optional string), the code being replaced; `replacement` (string), the new code; and `autoFixable` (boolean), indicating whether the fix can be applied without user review.

The `FileNode` interface models a file in the visualization graph. It includes `id` (string), `filePath` (string), `fileName` (string), `language` (string), `exports` (string array), `imports` (`FileImport` array), and `position` (optional x, y coordinates).

The `SecurityReport` interface aggregates scan results for export. It contains `id` (string), `projectName` (string), `generatedAt` (Date), `summary` (`ReportSummary`), `findings` (`ReportFinding` array), and `recommendations` (string array).

Services Package

The services package contains the classes that implement the business logic. Each service is designed to be instantiated once and shared across the application through dependency injection.

The `SecurityService` class orchestrates the vulnerability scanning process. It has a constructor that accepts an array of `SecurityScanner` instances. Its public methods include `scanFile(filePath: string, content: string): Promise<SecurityScanResult>`, which scans a single file using all available scanners in parallel; `scan(paths: string[], options?: SecurityScanOptions): Promise<SecurityScanResult>`, which scans multiple files or directories; `scanWithProgress(paths: string[], onProgress: ProgressCallback, options?:`

SecurityScanOptions): Promise<SecurityScanResult>, which provides streaming progress updates; getSupportedExtensions(): Promise<string[]>, which returns file extensions handled by the registered scanners; and getAvailableScanners(): Promise<string[]>, which lists the scanners that are currently operational.

The AutoFixService class manages the application and reversion of code modifications. It maintains a session-based change store that records all modifications, enabling undo functionality. Its methods include applyFixes(issues: CodeIssue[], options?: AutoFixOptions): Promise<{summary: FixSummary, sessionId: string}>, which applies fixes for multiple issues; revertFixes(sessionId: string): Promise<Result<number, string>>, which restores files to their pre-fix state; and formatSummary(summary: FixSummary): string, which generates a human-readable summary of fix operations.

The AIAnalysisService class provides AI-powered analysis capabilities. It has methods for explainIssue(issue: CodeIssue, codeContext: string): Promise<string>, which generates a natural-language explanation of a vulnerability; suggestFix(issue: CodeIssue, codeContext: string): Promise<CodeFix | null>, which asks the AI to propose a remediation; and classifyIssue(issue: CodeIssue): Promise<{severity: IssueSeverity, confidence: number}>, which uses AI to verify or adjust issue severity.

The ScanOrchestrator class coordinates the multi-tier scanning process. It manages the interaction between static scanners and AI agents, handles deduplication of findings, and produces a synthesized report. Its primary method is runFullAssessment(paths: string[], options?: AssessmentOptions): Promise<SynthesisReport>, which executes the complete analysis pipeline.

The ProviderRegistry class implements the registry pattern for managing adapters. It provides methods to register<T>(type: string, name: string, instance: T): void, get<T>(type: string, name?: string): T | undefined, and list(type: string): string[].

Adapters Package

The adapters package contains the implementations of the port interfaces.

The SemgrepScanner class implements the SecurityScanner port. It wraps the Semgrep command-line tool, parsing its JSON output into SecurityIssue objects. Its constructor accepts an execFn function for testability and an optional binaryPath for custom installations. It implements isAvailable(): Promise<boolean>, getSupportedExtensions(): string[], scanFile(filePath: string, content: string): Promise<SecurityIssue[]>, and scan(paths: string[]): Promise<ScanResult>.

The VercelAIAdapter class implements the AIProvider port using the Vercel AI SDK. It supports multiple providers through a factory pattern, creating the appropriate SDK client based on configuration. Its constructor accepts an AIProviderConfig object specifying the provider, model, and optionally an API key and base URL. It implements complete(prompt: string, context: AIContext): Promise<AIResponse>, stream(prompt: string, context: AIContext): AsyncIterable<string>, and isAvailable(): Promise<boolean>.

The `VSCodeFileSystem` class implements the `FileSystem` port using the VS Code workspace API. It provides methods for `readFile(path: string): Promise<string>`, `writeFile(path: string, content: string): Promise<void>`, `exists(path: string): Promise<boolean>`, `readDir(path: string): Promise<FileInfo[]>`, and `watch(path: string, callback: WatchCallback): () => void`.

Security Agents

The security agents form a hierarchy of specialized AI-powered analyzers. Each agent extends the `AbstractSecurityAgent` base class, which provides common functionality for AI communication, error handling, and progress reporting.

The `ChunkAnalystAgent` class analyzes individual code segments. It uses a specialized prompt template that instructs the AI to identify security vulnerabilities, logic flaws, and code quality issues within the provided code snippet. Its `analyze(context: AnalysisContext, onProgress?: ProgressCallback): AsyncIterable<SecurityIssue>` method yields issues as they are identified.

The `ComponentArchitectAgent` class examines entire modules and their interfaces. Its prompts focus on architectural security concerns such as improper exposure of internal state, missing access controls, and insecure default configurations.

The `FlowAnalysisAgent` class traces data flows across the codebase. It identifies paths where untrusted data can reach sensitive sinks without proper validation or sanitization. This agent is particularly effective at detecting injection vulnerabilities that span multiple files.

The `SentinelAgent` class synthesizes findings from all sources. It receives the aggregated issues from static scanners and other agents, groups them by root cause, assesses overall risk, and generates an executive summary. Its `synthesize(findings: RegisteredFinding[], context: AnalysisContext): Promise<SynthesisReport>` method produces the final report.

`BackBrain` is designed primarily as a runtime analysis tool, meaning that it does not maintain a traditional database for long-term storage. However, several aspects of the system require persistence across sessions, including user configuration, scan results cache, and fix history.

The persistence strategy employs a combination of VS Code's built-in storage mechanisms and local JSON files within the workspace. This approach aligns with the extension model, where each workspace has its own independent state.

Configuration Storage

User configuration is stored in the VS Code settings system, which persists to the user's `settings.json` file. The `BackBrain` extension contributes the following configuration keys: `backbrain.autoScan` (boolean, default true), which enables automatic scanning on file open and save; `backbrain.scanOnSave` (boolean,

default true), which triggers a scan when a file is saved; `backbrain.scanOnStartup` (boolean, default true), which runs a workspace scan when VS Code opens; `backbrain.preferredAIProvider` (string), which specifies the default AI provider; and `backbrain.severityThreshold` (string, default 'low'), which filters out issues below the specified severity.

Workspace State

The VS Code extension context provides a `workspaceState` object for storing workspace-specific data. This is used for the scan results cache, which stores the most recent scan results for each file to avoid redundant scanning. The cache is keyed by file path and includes the file's modification timestamp, so changes trigger re-scanning.

Fix History Storage

The fix history is stored in the extension's `globalState`, which persists across workspaces. Each fix operation records the session ID, timestamp, file paths modified, original content hashes, and applied fix descriptions. This enables the revert functionality to work even after VS Code is restarted, as long as the files have not been modified externally.

Vibe Rules Configuration

Custom vibe rules are stored in a JSON file at `.backbrain/vibe-rules.json` within the workspace root. This file is created by the extension on first activation if it does not exist. It contains an array of rule definitions, each specifying a pattern, severity, description, and optional autofix template. The extension watches this file for changes and reloads the rules automatically.

Data Model Diagram

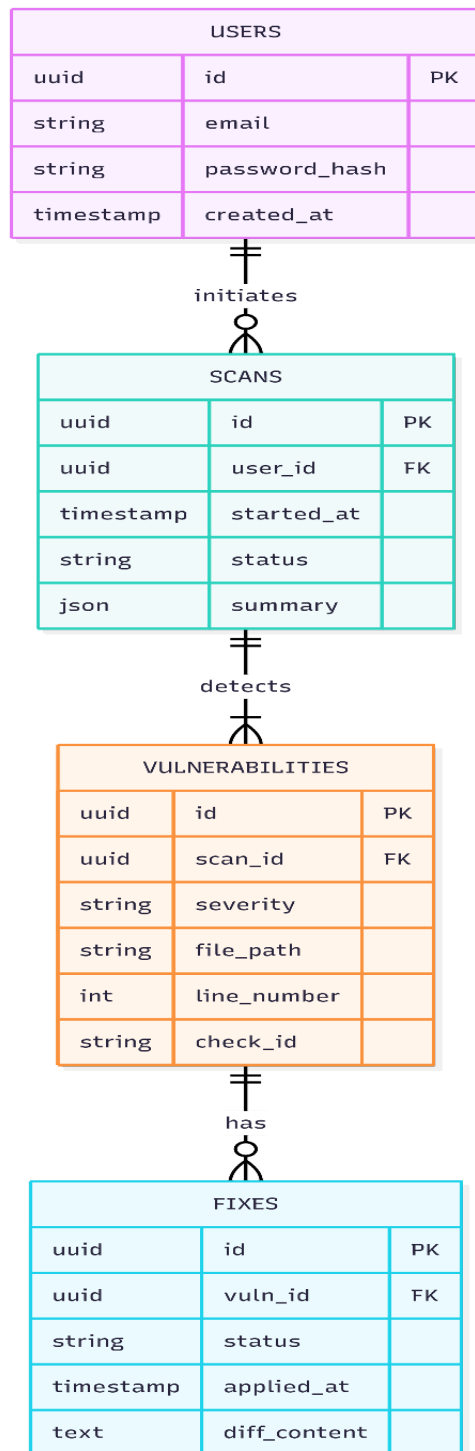


Figure 13: Persistent Data Model

The persistent data model diagram illustrates the structure of stored data. The Configuration entity is a singleton, stored in VS Code settings. The ScanCache entity has a one-to-many relationship with ScanResult, where each file path maps to at most one cached result. The FixHistory entity contains a list of FixSession records, each of which contains multiple FileChange records. The VibeRuleConfig entity holds an array of VibeRule objects.

The relationships are as follows: Configuration (1) configures () ScanCache; ScanCache (1) contains () ScanResult; FixHistory (1) contains () FixSession; FixSession (1) contains () FileChange; VibeRuleConfig (1) contains (*) VibeRule.

3.3 User Interface Design

The user interface of BackBrain is designed to integrate seamlessly into the developer's existing workflow. Rather than requiring context switches to external dashboards or web applications, all functionality is accessible directly within VS Code. The UI follows VS Code's design language, using native colors, icons, and interaction patterns to ensure a cohesive experience.

The interface is composed of four primary views: the Severity Panel, the Code Editor Integration, the Diff Preview, and the Node Canvas. Each view serves a distinct purpose and can be used independently or in combination.

Severity Panel

The Severity Panel is the central hub for viewing and acting on detected issues. It is implemented as a VS Code webview panel, rendered using React. The panel is registered with the viewId 'backbrain.severityPanel' and appears in the sidebar when the extension is activated.

The panel displays a list of issues grouped by severity. Each issue entry shows an icon indicating the severity level, the issue title, the file name and line number, and action buttons for "Explain", "Fix", and "Ignore". The list is sorted with critical issues at the top, ensuring that the most important problems are immediately visible. A search bar at the top allows filtering by keyword, and dropdown filters enable narrowing by severity, category, or file.

When the user clicks on an issue, the associated file opens in the editor with the cursor positioned at the issue location. If the file is already open, the editor scrolls to the relevant line and highlights it briefly to draw attention.

The panel also displays a summary section at the bottom, showing the total issue count broken down by severity. During a scan, this area transforms into a progress indicator showing the current phase, the file being analyzed, and the number of issues found so far.

Code Editor Integration

BackBrain integrates directly into the VS Code code editor through diagnostic decorations and hover providers. When issues are detected, the affected lines are decorated with colored squiggles: red for critical and high severity, yellow for medium, and blue for low and info. These decorations use the VS Code diagnostic system, ensuring consistency with other extensions and language servers.

When the user hovers over a decorated line, a hover tooltip appears with the issue title, a brief description, and a "Quick Fix" button. Clicking the quick fix button opens a code action menu with options to apply the suggested fix, explain the issue, or ignore it. This inline interaction model minimizes disruption to the coding flow.

For issues with available autofixes, a light bulb icon appears in the editor gutter. Clicking this icon or pressing the quick fix keyboard shortcut (Ctrl+. or Cmd+.) opens the same code action menu.

Diff Preview

Before applying a fix, users can preview the proposed changes in a Diff Preview panel. This panel uses VS Code's built-in diff editor to show a side-by-side comparison of the original code and the modified code. The left pane shows the current file content, and the right pane shows the result after applying the fix.

The diff preview is non-modal, allowing users to review the changes while continuing to work. An "Apply Fix" button at the top of the panel confirms the change, while a "Cancel" button dismisses the preview without modification. If multiple fixes are available for the same file, the preview shows the cumulative effect of all selected fixes.

Node Canvas

The Node Canvas provides a visual representation of the codebase architecture. It renders files and folders as nodes in an interactive graph, with edges representing import and export relationships. This visualization helps developers understand the structure of their project and identify potential architectural issues.

The canvas is built using React Flow, a library for building node-based user interfaces. Nodes are styled to indicate file types: TypeScript files have a blue header, JavaScript files have yellow, Python files have green, and configuration files have gray. The size of a node reflects the file's line count, giving a visual sense of code complexity.

Users can interact with the canvas by dragging to pan, scrolling to zoom, and clicking nodes to select them. Selecting a node highlights its direct dependencies in a contrasting color and dims unrelated nodes. Double-clicking a node opens the corresponding file in the editor.

An overlay panel on the canvas shows summary statistics: total files, total connections, average connections per file, and a list of the most connected files (potential hotspots). The canvas also supports filtering to show only files with detected issues, making it easy to identify problem areas.

User Interface Mockups

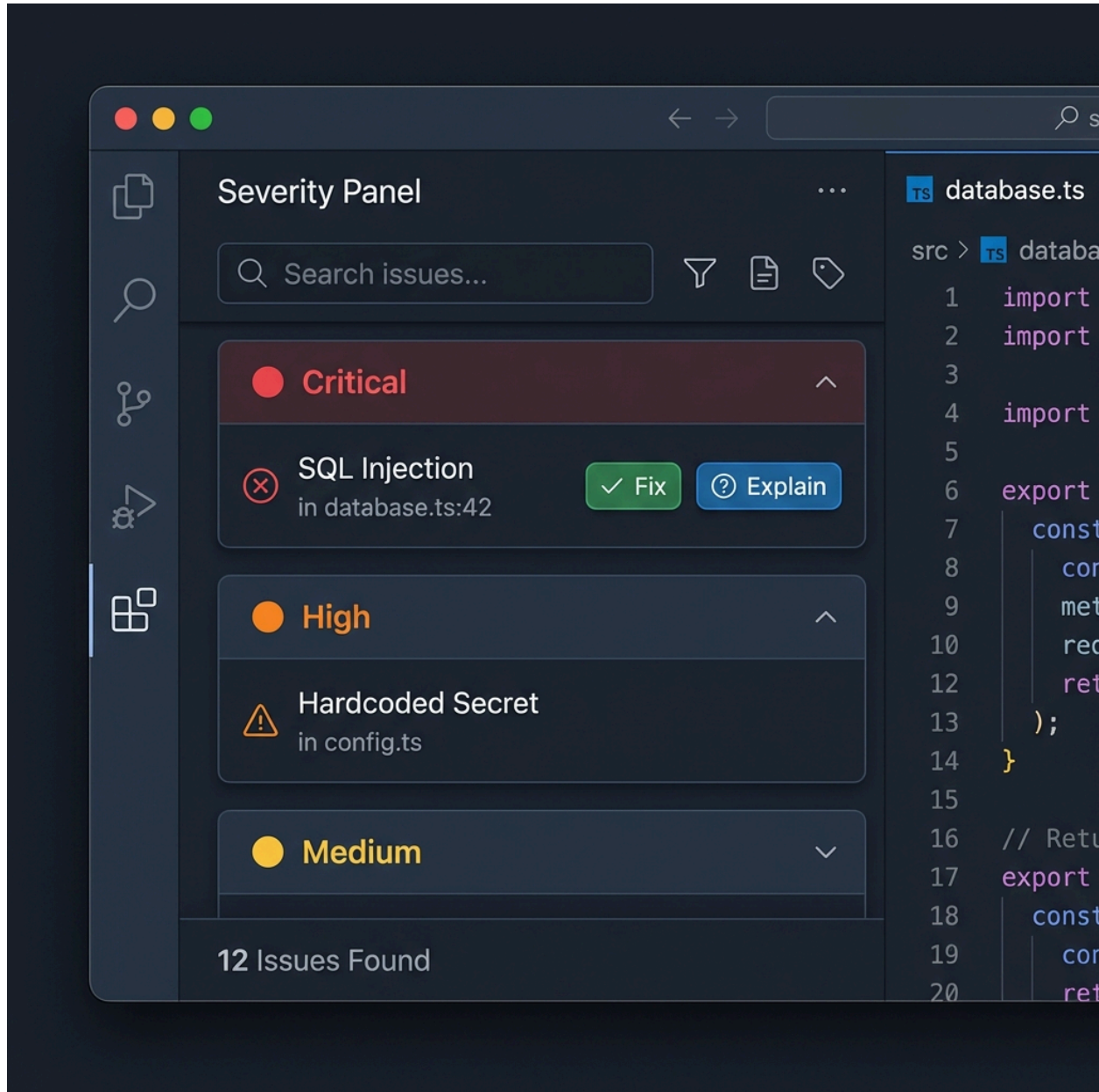


Figure 14: Severity Panel Mockup

The Severity Panel mockup shows the issue list grouped by severity, with critical issues in a red section, high in orange, medium in yellow, and low in blue. Each entry displays an icon, title, file location, and action buttons.

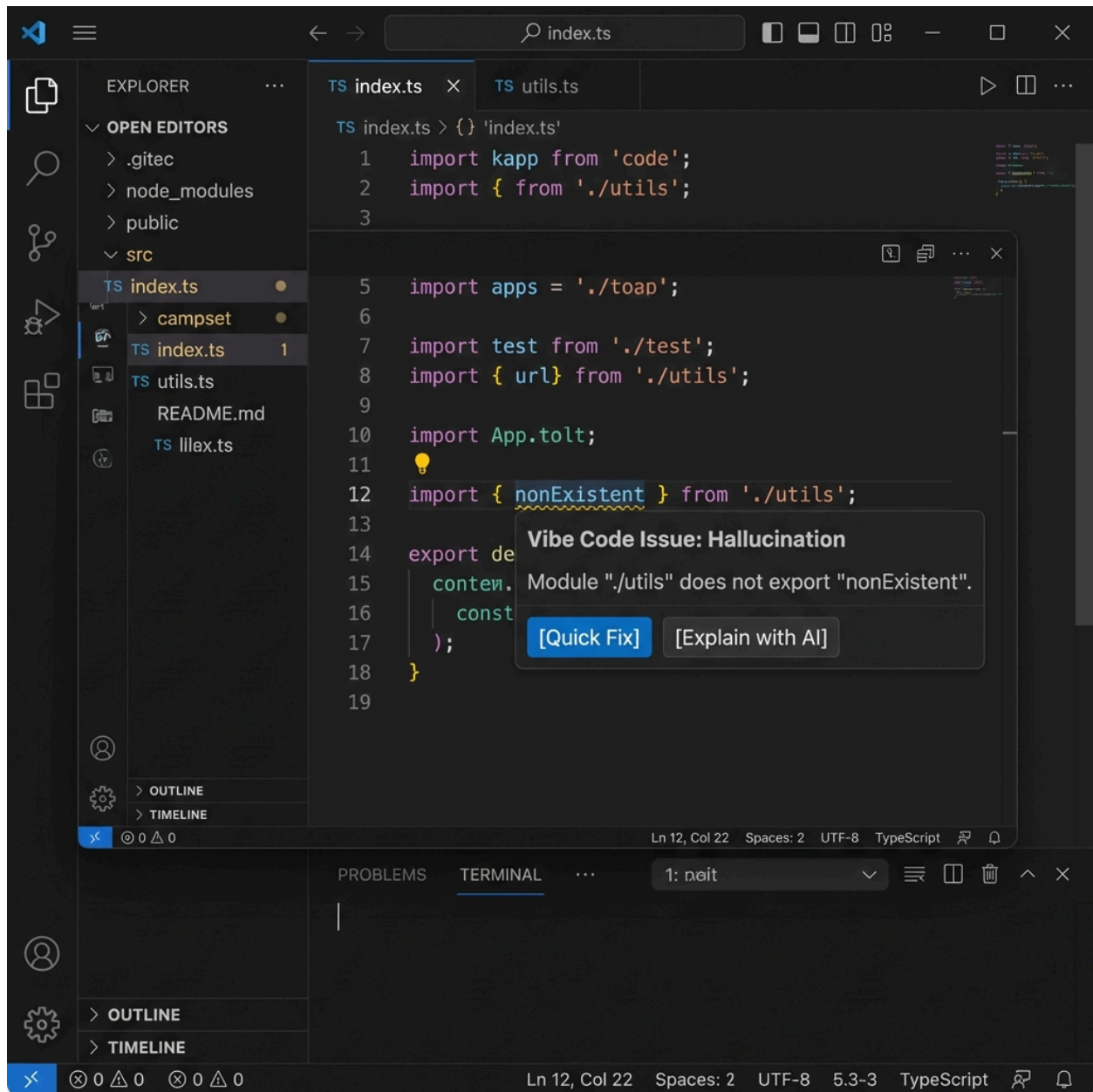


Figure 15: Code Editor Integration Mockup

The Code Editor Integration mockup demonstrates the inline decorations and hover tooltip. A yellow squiggle under a suspicious line indicates a medium-severity issue, and the hover popup shows the issue details with quick action buttons.

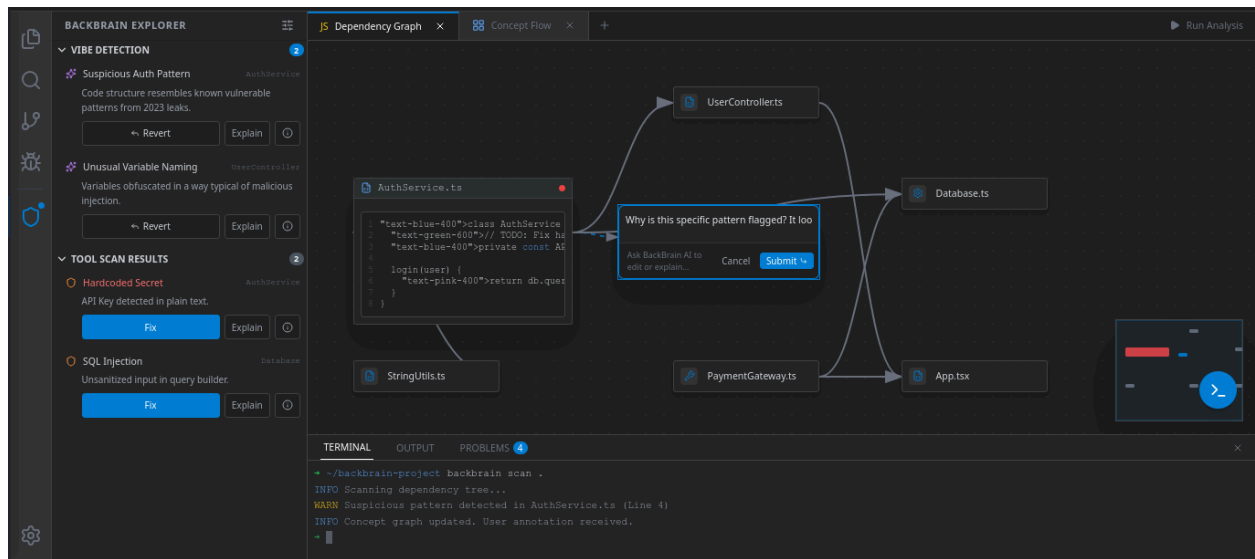


Figure 16: Node Canvas Mockup

The Node Canvas mockup presents the file graph with several nodes representing source files. Edges connect nodes that have import relationships. A selected node is highlighted with its dependencies shown in a brighter color.

3.4 Access Control and Security

As a security tool, BackBrain must itself adhere to stringent security practices. This section describes the access control mechanisms, data handling policies, and security measures implemented in the system.

Authentication and Authorization

BackBrain operates within the user's local development environment and does not implement its own authentication system. Access to the extension's features is implicitly authorized by the user's access to VS Code and their local file system. There are no user accounts or roles within BackBrain itself.

However, access to AI features requires API keys for the respective providers. These keys are stored securely using VS Code's secret storage API, which leverages the operating system's credential manager (Keychain on macOS, Credential Manager on Windows, and libsecret on Linux). The keys are never written to plain text files or exposed in logs.

Data Handling and Privacy

User code is treated as highly sensitive data. The system adheres to the following principles: local-first processing, where all static analysis (Sempgrep, Gitleaks, Tree-sitter, Vibe-code rules) runs entirely on the

user's machine with no data leaving the local environment; opt-in AI analysis, where code is only sent to external AI providers when the user explicitly requests AI-powered features such as "Explain Issue" or "Suggest Fix"; minimal data transmission, where when AI analysis is triggered only the relevant code snippet and surrounding context are sent, not the entire file or project; and no persistent storage on AI servers, where the integration uses stateless API calls with no conversation history maintained on the provider side.

The extension does not collect telemetry or usage data unless the user opts in through VS Code's telemetry settings. If telemetry is enabled, only anonymous, aggregated metrics are collected, such as the number of scans performed and the types of issues detected, without any code content.

Secure Coding Practices

The BackBrain codebase follows secure coding practices enforced through multiple mechanisms. Static analysis is performed using Semgrep on the BackBrain code itself as part of the CI/CD pipeline, ensuring that the security tool does not contain the very vulnerabilities it detects in user code. Dependency scanning is conducted using automated tools that check for known vulnerabilities in third-party packages, with alerts triggering immediate review and updates. Type safety is ensured through strict TypeScript configuration with no implicit any types, catching many potential issues at compile time. Input validation is applied to all external inputs, including file paths and AI responses, which are validated before use.

Threat Model

The threat model for BackBrain considers the following potential attack vectors: malicious code in scanned projects, where an attacker could craft code that exploits vulnerabilities in the scanning tools, which is mitigated by running scanners in isolated processes with limited privileges; supply chain attacks, where compromised dependencies could introduce malicious code, which is mitigated by using lock files, verifying package integrity, and regularly auditing dependencies; API key theft, where if the user's workstation is compromised an attacker could extract API keys, which is mitigated by using the OS credential manager and never exposing keys in logs or UI; and prompt injection, where malicious code could attempt to manipulate AI behavior through crafted prompts, which is mitigated by sanitizing code content before inclusion in prompts and validating AI responses.

AI Strategy and Prompt Engineering

A critical aspect of BackBrain's security posture is its approach to AI integration. Rather than training custom models from scratch, which would require massive computational resources and extensive security-focused training data, the system leverages existing large language models through sophisticated prompt engineering. This strategy offers several advantages: access to state-of-the-art reasoning capabilities from models like GPT-5 and Claude; rapid adaptation to new vulnerability patterns by

updating prompts rather than retraining; reduced risk of model-specific vulnerabilities; and flexibility to switch providers based on performance, cost, or availability.

The prompt engineering strategy involves several key techniques. Structured system prompts establish the AI's role as a security analyst, defining the expected response format and the criteria for identifying issues. Few-shot examples provide the AI with annotated examples of code vulnerabilities and proper analysis output, improving the quality and consistency of responses. Signature-based augmentation feeds the AI a library of known vulnerability signatures and patterns specific to AI-generated code, enabling detection of issues like phantom dependencies or logic drift. Response validation parses and validates all AI responses, rejecting malformed output and falling back to static analysis results when AI confidence is low.

The system prompts are versioned and tested against a suite of known vulnerable code samples. When a new prompt version is developed, it is evaluated on this test suite to ensure it maintains or improves detection accuracy before deployment.

Future Evolution: Fine-Tuning Strategy

While the current system relies on prompt engineering with general-purpose models, the long-term roadmap includes fine-tuning specialized models once sufficient data and resources are available. The plan involves several phases. In the first phase, prompt-only operation is used during initial launch and user base growth, relying entirely on prompt engineering with off-the-shelf models. In the second phase, data collection occurs once the user base reaches a critical mass, with anonymized and aggregated patterns from detected issues collected with user consent to build a training dataset. In the third phase, fine-tuning is performed using the collected data to fine-tune a smaller, faster model specifically for security analysis, reducing API costs and latency. In the fourth phase, on-device models are deployed, deploying the fine-tuned model locally within the extension, eliminating the need for external API calls and enabling fully offline AI-powered analysis.

This phased approach ensures that the system can deliver value from day one while building toward a more powerful and cost-effective solution as the user base and resources grow. The architecture's clean separation between the AIProvider port and its adapters means that the fine-tuned model can be integrated as a new adapter without any changes to the core analysis logic.

IDE Migration Security Considerations

The planned migration to a standalone IDE introduces additional security considerations. The standalone application will have direct file system access without the sandboxing provided by VS Code's extension model. To maintain security, the IDE will implement permission-based file access, requiring explicit user approval for operations outside the workspace; process isolation, running scanners and AI agents in separate processes with minimal privileges; code signing, using signed application bundles to ensure

integrity and prevent tampering; and auto-update verification, validating update packages before installation to prevent supply chain attacks.

These measures ensure that BackBrain remains a trusted component of the developer's toolchain, whether running as an extension or as a standalone IDE.

References

- [1] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/> [Accessed: Oct. 14, 2025].
- [2] Semgrep, Inc., "Semgrep Documentation," 2024. [Online]. Available: <https://semgrep.dev/docs/> [Accessed: Dec. 01, 2025].
- [3] Microsoft, "Visual Studio Code Extension API," VS Code Docs. [Online]. Available: <https://code.visualstudio.com/api> [Accessed: Nov. 18, 2025].
- [4] Vercel, "Vercel AI SDK Documentation," 2024. [Online]. Available: <https://sdk.vercel.ai/docs> [Accessed: Jan. 17, 2026].
- [5] @mr_vibe_it, "Technical Analysis of LLM Security," X, Jan. 2026. [Online]. Available: https://x.com/mr_vibe_it/status/2013741354719781070 [Accessed: Jan. 15, 2026].
- [6] G. Huntley, "The Ralph Wiggum Technique: Autonomous AI Agent Orchestration," 2024. [Online]. Available: <https://ghuntley.com/loop/> [Accessed: Jan. 25, 2026].
- [7] G. Huntley, "RALPH: Autonomous AI Agent Loop," GitHub Repository, 2024. [Online]. Available: <https://github.com/snarktank/ralph> [Accessed: Jan. 25, 2026].
- [8] J. Li, Q. Zhang, Y. Yu, Q. Fu, and D. Ye, "More Agents Is All You Need," *arXiv preprint arXiv:2402.05120*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.05120> [Accessed: Jan. 25, 2026].

Appendices

Appendix A: Sample Code Issue JSON Data Structure

Document Title: BackBrain Standardized Issue Schema

Format Version: 1.2

Type: Technical Data Specification

Description: This appendix defines the JSON structure used by the BackBrain engine to communicate vulnerabilities. It includes metadata for issue identification, severity mapping, and the "suggestedFix" logic.

```
{
  "id": "issue_12345",
  "type": "security_vulnerability",
  "title": "SQL Injection Vulnerability",
  "description": "Detected a potential SQL injection vulnerability where
user input is directly concatenated into a query string.",
  "location": {
    "filePath": "packages/core/src/database.ts",
    "line": 42,
    "column": 15
  },
  "severity": "critical",
  "category": "security",
  "suggestedFix": {
    "description": "Use parameterized queries to prevent SQL injection.",
    "original": "const query = `SELECT * FROM users WHERE id = ${id}`;",
    "replacement": "const query = 'SELECT * FROM users WHERE id =
?';\nconst params = [id];",
    "autoFixable": true
  }
}
```

Appendix B: Glossary of Specialized Terms

Document Title: Glossary Terms

Description: This glossary provides formal definitions for terms that are used but never or loosely defined

- **Vibe Coding:** A development style where the developer relies heavily on AI

agents to generate code, often prioritizing speed and high-level intent over manual line-by-line implementation.

- **Hallucination:** In the context of LLMs, this refers to the generation of code that is syntactically correct but refers to non-existent libraries, functions, or variables.
- **Autofix:** The capability of the system to not only detect an issue but also generate and apply a code patch to resolve it automatically.
- **Node-Based Visualization:** A graphical representation of the codebase where files are nodes and relationships (imports/calls) are edges.
- **Security Guardian:** The core engine of BackBrain that proactively monitors code for vulnerabilities and quality issues.
- **Security-Velocity Gap:** is the critical disconnect where AI-accelerated development creates code faster than traditional security tools can scan and fix it, leaving a growing window of unaddressed vulnerabilities.

Appendix C: Prompt Template

Document Title: AI Agent finetuning or Prompt Templates

Version: 0.5(early concept version)

Description: This json file depicts the initial idea or version of the prompt that would planned to given the ai to finetune it or optimize it to specialize vulnerable scanning

```
{  
  
  "agent_configuration": {  
  
    "role": "Senior Security Remediation Engineer",  
  
    "specialization": "Vibe-coded applications (rapid prototype, AI-generated)",  
  
    "objective": "Identify logic flaws and hallucinated dependencies, rank by impact, and  
generate safe patches."  
  
  },  
  
  "context_inputs": {  
  
    "signatures": "[INJECT_SIGNATURES_JSON_HERE]",  
  
    "semgrep_results": "[INJECT_SEMGREP_JSON_HERE]",  
  
    "gitleaks_results": "[INJECT_GITLEAKS_JSON_HERE]",  
  
    "target_codebase": "[INJECT_CODE_SNIPPET_HERE]"  
  
  }  
}
```

```

    },
    "execution_steps": [
        {
            "step_id": 1,
            "action": "identify_tech_stack",
            "instruction": "Analyze the codebase to determine language, framework, and package manager. Watch for mixed stacks."
        },
        {
            "step_id": 2,
            "action": "cross_reference_analysis",
            "instruction": "Compare scanner findings against the injected signatures. Identify 'ghost' dependencies or vibe-specific logic gaps."
        },
        {
            "step_id": 3,
            "action": "generate_threat_model",
            "output_format": "markdown_table",
            "columns": ["Rank", "Vulnerability_Type", "File_Path", "Impact_Score", "Remediation_ID"],
            "logic": "Rank = Critical if finding matches High-Severity signature AND exists in active code path."
        },
        {

```

```

    "step_id": 4,

    "action": "plan_remediation",

    "target": "Top-ranked vulnerability from Step 3",

    "requirements": [

        "Define backup filename (e.g., .vuln.bak)",

        "Generate fix code block preserving existing coding style",

        "Explain verification method"

    ]

}

]

}

{
  "agent_metadata": {
    "role": "Lead Security Remediation Engineer",
    "persona_alignment": "Expert-level AppSec specialist with high-fidelity reasoning in AI-generated code patterns.",
    "primary_objective": "Autonomous detection, prioritization, and remediation of logic flaws and supply-chain anomalies."
  },
  "knowledge_context": {
    "proprietary_signatures": "[INJECT_VIBE_SIGNATURES_JSON]",
    "static_analysis_telemetry": {
      "semgrep_findings": "[INJECT_SEMGREP_JSON]",
      "secret_leaks": "[INJECT_GITLEAKS_JSON]"
    },
    "source_code_inventory": "[INJECT_TARGET_CODE_BLOCKS]"
  },
  "operational_pipeline": [
    {
      "phase": "Stack_Discovery",
      "instruction": "Statically analyze the provided source code to map the technology

```

stack (languages, frameworks, and transitive dependencies). Identify discrepancies between declared and utilized packages to flag potential 'ghost' or hallucinated dependencies."

```
    },
    {
      "phase": "Signature_Heuristics",
      "instruction": "Correlate telemetry from Semgrep and Gitleaks with the 'proprietary_signatures' database. Specifically look for insecure 'vibe-coded' patterns: hardcoded testing placeholders, disabled CSRF/CORS protections, and unvalidated AI-generated logic."
    },
    {
      "phase": "Risk_Quantification",
      "output_artifact": "threat.md",
      "logic": "Generate a prioritized threat matrix. Score each finding based on exploitability within the identified tech stack and business impact (P0-P3).",
      "format": [ "Rank", "Vulnerability_Signature", "File_Path", "Severity_Score", "Logic_Justification" ]
    },
    {
      "phase": "Autonomous_Remediation",
      "execution_logic": "For the highest-ranked threat, implement a three-stage fix: 1. Create an immutable backup ({filename}.vuln.bak). 2. Draft a context-aware patch that adheres to the original code's functional 'vibe' while closing the security gap. 3. Output a verification summary detailing why the fix is secure."
    }
  ],
  "agent_constraints": {
    "integrity": "Do not alter business logic beyond the scope of the security patch.",
    "backup_protocol": "A backup file must be confirmed before any source code modification.",
    "hallucination_filter": "Verify the existence of any library suggested in the remediation plan."
  }
}
```

Appendix D: Planning and Threat Modeling

Document Title: Proactive Security Consulting Framework

Process Phase: Pre-Development & Verification

Objective: Logical Security Alignment

Description: This appendix describes the profiling method used to generate security checklists based on application frameworks and data sensitivity. It outlines the verification logic used by agents to ensure AI-generated features do not bypass established security protocols.

The system incorporates a proactive security consulting phase that begins with product profiling to identify frameworks, authentication methods, and data sensitivity. Based on this profile, the system generates a comprehensive security checklist that defines the required security controls for the specific application type. During the verification phase, the AI agents check the implementation against these requirements to ensure that the codebase remains consistent with the intended security architecture. This process is particularly effective for catching logical security flaws that occur when AI agents add features that work correctly but bypass established security protocols or use insecure default configurations.

Appendix E: BackBrain Project Documentation

Document Title: BackBrain Internal Technical Manual

Version: 1.0

Description: This appendix contains the internal documentation for the BackBrain system architecture, API specifications, and security protocols developed during this project.

Note: For the full digital repository, please refer to the [docs/](#) folder within the submitted project source code.